

信息科学与工程学院
课程设计报告

课程名称： 硬件技术基础课程设计

题目： 单周期 CPU 逻辑设计

年级/专业： 22 计算机科学与技术

学生姓名： 顾凯琪

学号： 226001624

指导老师： 严迪群、钱江波

开始时间：2024 年 9 月 17 日

结束时间：2024 年 10 月 31 日

摘　要

本文详细探讨了一种基于单周期设计的中央处理器（CPU）的指令集架构。该 CPU 支持十一种指令，每种指令长度为 32 位，包括 ADD、LW、SW、J、SHL、CMP、SUB、MOV、ADDN、SUBN、CLEAR。针对每一条指令，文中表现了其具体功能，给出了逻辑转换图和指令处理相关器件的执行过程图。CPU 的执行流程分为取指令、指令译码、指令执行、存储器访问和结果写回等五个阶段。相应的控制信号在各个阶段中也被明确定义。我们还在文中提供了各个器件单元的详细设计图和内部线路图，包含 PC 程序计数器、寄存器堆、控制器、ALU 运算器、存储器、扩展单元和时钟扩展单元等核心组件。最后，文中展示了我们为确保 CPU 设计的正确性和性能在 DE0 板上的测试方案，测试过程，测试结果，以及我们在单周期 CPU 设计方面的心得体会。这一研究为初步理解和实现 CPU 设计提供了详实的指导。

关键词：单周期 CPU 设计，指令集架构，控制信号，测试验证，DE0 开发板

Abstract

This paper provides an in-depth exploration of a single-cycle Central Processing Unit (CPU) based on a custom instruction set architecture. The CPU supports eleven 32-bit instructions, including ADD, LW, SW, J, SHL, CMP, SUB, MOV, ADDN, SUBN, and CLEAR. For each instruction, the paper details its specific functionality, presents logic conversion diagrams, and illustrates the execution process in the relevant components. The CPU's execution flow is divided into five stages: instruction fetch, decode, execution, memory access, and write-back, with corresponding control signals clearly defined at each stage. Detailed design and internal circuitry diagrams for each component are provided, including the program counter (PC), register file, controller, Arithmetic Logic Unit (ALU), memory, extension units, and clock extension units. Finally, a testing scheme, process, and results on the DE0 board are presented to verify the CPU's design correctness and performance. This study offers comprehensive guidance for understanding and implementing a single-cycle CPU design.

Keywords: single-cycle CPU design, instruction set architecture, control signals, testing and validation, DE0 development board

目　录

一、设计目的与目标 1

1.1 设计目的 1

1.2 设计目标 1

二、课程设计器材 3

2.1 硬件平台 3

2.2 软件平台 3

三、CPU 逻辑设计总体方案 4

3.1 指令模块 4

3.2 器件单元模块 6

四、模块详细设计 7

4.1 指令设计模块 7

4.2 器件单元模块 14

五、测试结果及分析 32

5.1 测试方案 32

5.2 模块初始值 32

5.3 测试结果及分析 34

六、心得与体会 37

参考文献 38

一、设计目的与目标

1.1 设计目的

通过设计硬布线控制器，我们能够掌握以下关键概念和技能：

1. **指令设计：** 掌握设计有效指令集的方法，熟悉指令格式与操作码的合理分配。
2. **硬件模块设计：** 学习如何设计硬布线控制器所需的逻辑电路模块，涵盖组合逻辑电路与触发器等元件，以实现多样化的控制信号。
3. **时钟设计：** 理解机器周期、节拍和主频的概念，能够合理确定每个周期的操作及所需周期数。
4. **Quartus II 开发软件的使用：** 熟练运用 Quartus II 软件进行设计、仿真与实现，能够通过该工具进行逻辑表达式的综合、优化，并生成所需的控制信号。
5. **逻辑设计思想：** 领会硬布线控制器逻辑设计中的关键要点，包括指令综合、逻辑电路选择及控制信号设计的最佳实践。
6. **比较硬布线控制器与微程序控制器：** 掌握两者在实现方式与性能方面的差异，能够在不同应用场景下选择合适的控制器。

通过本实验，我们将深入理解计算机硬件控制的基础原理，提升硬件设计的实践能力及解决问题的能力。同时，还将熟练掌握 Quartus II 等开发工具的使用，为未来数字系统设计及嵌入式系统开发奠定坚实的基础。

1.2 设计目标

本实验的主要目标是通过在 The DE0 board 上测试单周期指令设计，验证指令的正确执行和硬件设计的有效性。具体目标如下：

1. **指令功能验证：**
    - 验证 ADD 指令，确保能够正确将两个寄存器的值相加，并将结果存储到指定的寄存器中。
    - 验证 LW 指令，确认可以从存储器中正确加载数据，并将其存入目标寄存器。
    - 验证 SW 指令，确保可以将指定寄存器中的值正确存储到存储器的指定地址。
    - 验证 J 指令，确认能够正确跳转到指定的地址并执行指令。
    - 验证 SHL 指令，确认可以将指定寄存器中的数据左移一位，并将结果正确存储到目标寄存器中。
    - 验证 CMP 指令，确认能够正确比较两个指定寄存器中的数值，并将比较结果按照规定的编码存入目标寄存器中。
    - 验证 SUB 指令，确保能够正确将一个寄存器的值减去另一个寄存器的值，并将结果存储到指定的寄存器中。
    - 验证 MOV 指令，确认可以将一个寄存器中的值正确复制到另一个寄存器中。
    - 验证 ADDN 指令，确认可以将一个寄存器的值与一个符号扩展后的 16 位偏移量相加，并将结果存入目标寄存器中。
    - 验证 SUBN 指令，确认可以将一个寄存器的值与一个符号扩展后的 16 位偏移量相减，并将结果存入目标寄存器中。

- 验证 CLEAR 指令，确认能够将指定的寄存器的值设置为 0。

2. **硬件模块验证：**
   - **确保 ALU**：验证算数逻辑单元（ALU）是否能够正确执行 ADD、SHL、CMP、SUB 等算术和逻辑运算指令，确保运算结果准确无误。
   - **验证寄存器堆**：检查寄存器堆的功能，确保数据能够正确地被读取和写入，保证寄存器间的数据流动正常。
   - **验证存储器**：确保存储器能够准确地进行数据的读写操作，保证数据在存储器中的操作不会出错，确保系统的整体稳定性。
3. **Quartus II 软件验证：**
   - **使用 Quartus II 软件仿真硬布线控制器**：通过 Quartus II 软件对设计的硬布线控制器进行仿真，确保在模拟环境中各条指令都能按预期正确执行，验证功能的正确性。
   - **进行时序分析**：对设计进行详细的时序分析，确保整个设计能够满足时序要求，避免由于时序问题导致的不稳定或功能失效。
4. **The DE0 board 实验验证：**
   - 在 The DE0 board 上实际加载程序，通过观察 LED 灯显示的数据来验证指令的执行结果。
   - 通过按钮或开关等输入设备触发不同指令的执行，验证交互性。
5. **性能评估：**
   - 记录每条指令的执行时间，评估硬布线控制器的性能。
   - 分析硬布线控制器与微程序控制器的差异，探讨在不同场景中的适用性。
6. **设计修正：**
   - 如果在测试中发现指令执行错误或硬件设计存在缺陷，及时进行修正。
   - 优化硬件设计，提高性能和可维护性。

通过实现这些目标，我们将全面掌握计算机硬布线控制器的设计流程和实际硬件验证技能。同时，我们也会熟练使用 Quartus II 软件，进行数字系统设计、仿真和调试。这不仅帮助我们在理论和实践上都打下坚实的基础，还将为未来从事数字系统设计和嵌入式系统开发提供宝贵的经验和有力的支持。

二、课程设计器材

2.1 硬件平台

PC机两台、The DE0 board（编号：0545）

2.2 软件平台

Quartus II硬件设计平台

三、CPU 逻辑设计总体方案

3.1 指令模块

为了使 CPU 能够正常的运行，我们总共设计了八条指令，每条指令总共 32 位，前六位表示指令译码，后面的位数可能表示寄存器的地址，也可能表示偏移量，接下来我们来具体设计每一条指令。

(1) ADD rs, rt, rd, reserved

ADD 000000	rs(5位)	rt(5位)	rd(5位)	reserved(11位)

完成功能：[rs]+[rt]->rd。

(2) LW base, rt, offset

LW 100011	base(5位)	rt(5位)	offset(16位)

完成功能：16 位 offset 经符号扩展扩展到 32 位，与寄存器 base 中值相加，结果作为地址，以此地址取字保存到寄存器 rt。

(3) SW base, rt, offset

SW 101011	base(5位)	rt(5位)	offset(16位)

完成功能：16 位 offset 经符号扩展到 32 位，与寄存器中 base 中值相加，结果作为地址，寄存器 rt 值保存到此地址内存中。

(4) J target

J 000010	target(26位)

完成功能：26 位地址经 0 扩展成 32 位，作为跳转目标地址。

(5) SHL rs, rt, reserved

SHL 100001	rs(5位)	rd(5位)	reserved(16位)

完成功能：rs 里的数据左移一位放到 rd。

(6) CMP rs, rt, rd, reserved

CMP 101001	rs(5位)	rt(5位)	rd(5位)	reserved(11位)

完成功能：比较[rs][rt]内的数值，rs 大，后三位为 100，一样大，后三位为 010，rt 大，后三位为 001，前面为 0，放到 rd。

(7) SUB rs, rt, rd, reserved

SUB 100111	rs(5位)	rt(5位)	rd(5位)	reserved(11位)

完成功能：[rs]-[rt]->rd

(8) MOV rs, rd, reserved

MOV 111111	rs(5位)	rd(5位)	reserved(16位)

完成功能：[rs]->[rt]

(9) ADDN

ADDN 100010	rs(5位)	rd(5位)	offset(16位)

完成功能：16 位 offset 经符号扩展扩展到 32 位，与寄存器 rs 中值相加，将结果送入 rd 寄存器内

(10) SUBN

SUBN 101111	rs(5位)	rd(5位)	offset(16位)

完成功能：16 位 offset 经符号扩展扩展到 32 位，与寄存器 rs 中值相减，将结果送入 rd 寄存器内

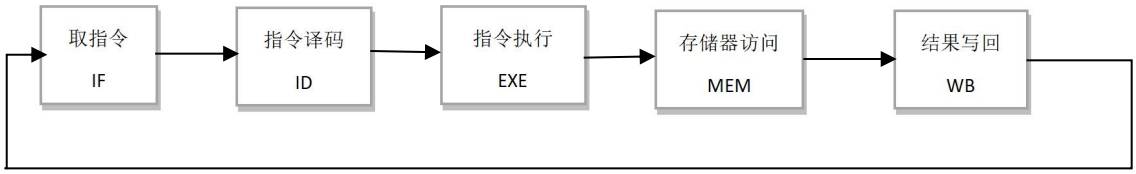
(11) CLEAR

CLEAR 000001	reserved(5位)	reserved(5位)	rd(5位)	reserved(11位)

完成功能：[rd]->0

一般来说，一个 CPU 在处理指令时需要经过以几个步骤：

1) 取指令(IF)：根据程序计数器 PC 中的指令地址，从存储器中取出一条指令，然后转到译码状态。同时，在 PC 中产生取下一条指令需要的指令地址。
2) 指令译码(ID)：对取指令操作中得到的指令进行译码，确定这条指令需要完成的操作，从而产生相应的控制信号，驱动执行状态中的各种动作。
3) 指令执行(EXE)：根据指令译码得到的控制信号，具体地执行指令动作，然后转移到结果写回状态。
4) 存储器访问(MEM):所有需要访问存储器地操作都将在这个步骤中执行，该步骤给出访问存储器的数据地址，把数据写入到存储器中数据地址所指示的位置或者从存储器中的得到数据地址所指示的数据。
5) 结果写回（WB）：该步骤负责把指令执行的结果或者访问存储器中得到的数据写回到相应的目的寄存器中。



控制信号作用表

信号名	无效时作用(0)	有效时作用(1)
PCSource	PC+1	拓展的立即数
MenWr	无	时钟上升沿时，写存储器
MemtoReg	送往寄存器堆写数据输入的值来自ALU	送往寄存器堆的数值来自内存
RegDst	写寄存器对在寄存器堆的地址来自于rt字段	写寄存器堆在寄存器堆的地址来自于rd字段
RegWrite	无	时钟上升沿到来时，写寄存器
ALUSrcB	寄存器堆Data2输出	经符号扩展的立即数
ALUOP	无	加法
SHLOP	无	逻辑左移
SUBOP	无	减法
CMPOP	无	比较输出大数

3.2 器件单元模块

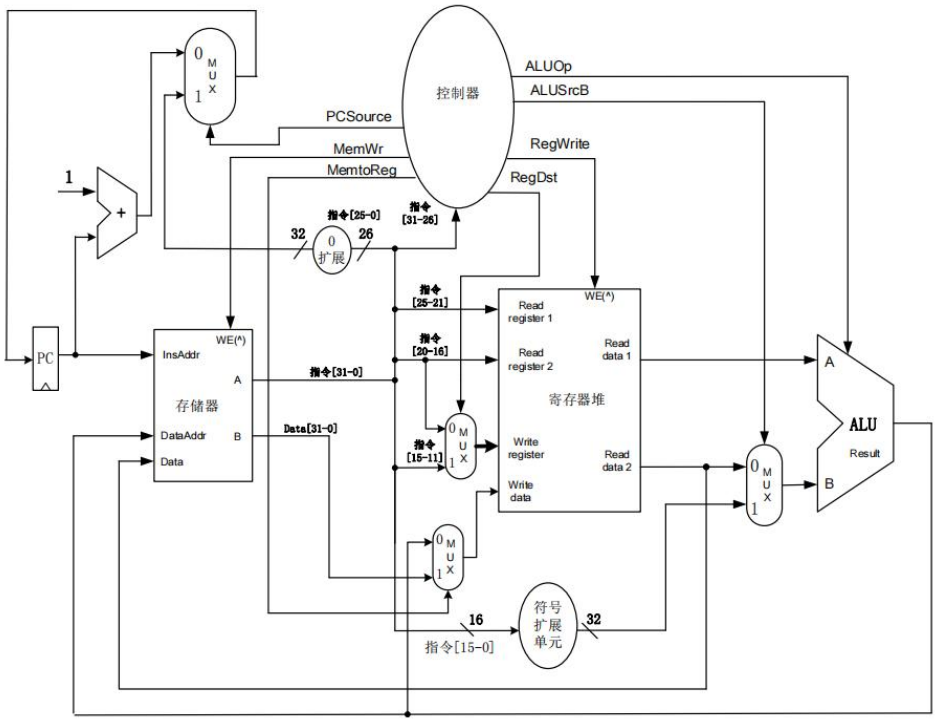


图 3.2-1：单周期 CPU 数据通路图和控制线路图

图 3.2-1 是一个非常简单的能够单周期完成上述 4 条指令的数据通路和必要的控制线路图。构建一个硬布线控制的 CPU 大概需要如下组件的协调运行

(1) PC 程序计数器

程序计数器，用来实现指令的跳转，位 CPU 提供程序指令。

(2) 寄存器堆

由多个 32 位的寄存器组成，用来可实现双地址读写操作，用以临时存储数据

(3) 控制器

用于对指令进行译码，输出对应的控制信号来驱动执行状态中的各种动作。

(4) ALU 运算器

用于实现指令中包含的各种运算功能，运算方式由控制信号给出。

(5) 存储器

由两个存储器组成，ROM 存储器存储指令，RAM 存储器存储数据，通过 PC 或地址总线提供的地址输出对应的数据

(6) 扩展单元

包括符号扩展与 0 扩展，实现高位补符号位以及高位补 0 的操作。

(7) 时钟扩展单元

为了让元器件按照时序有序运行，我们将 1 个时钟信号扩展为 5 个时钟信号，然后将其按照时序输入到不同的器件当中。

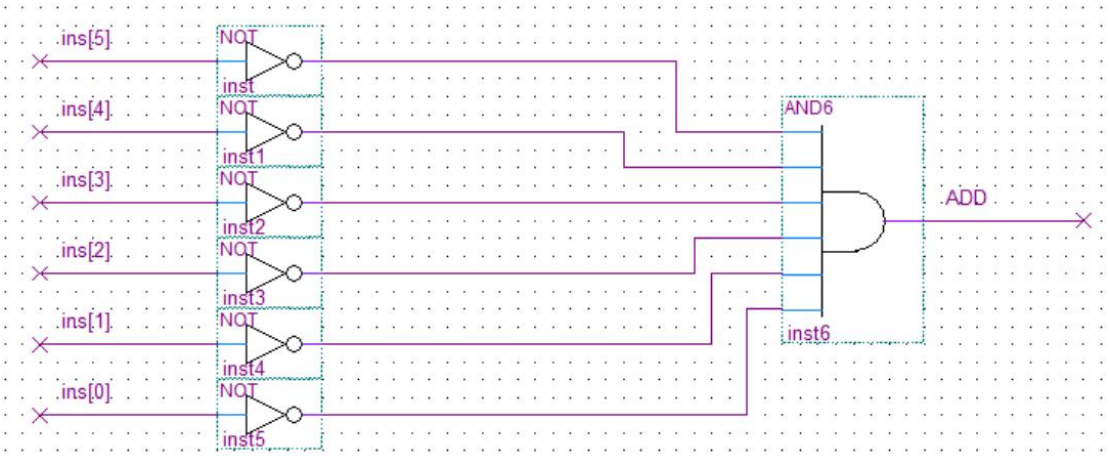
四、模块详细设计

4.1 指令设计模块

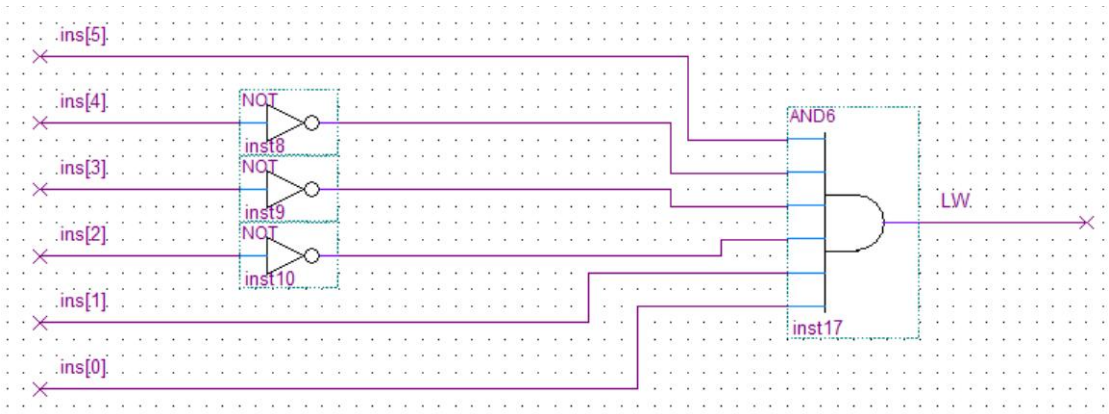
控制信号转化表

信号 / 指令码	PCSource	MemWr	MemtoReg	RegDst	RegWrite	ALUSrcB	ALUOP	SHLOP	SUBOP	CMPOP
ADD 000000	0	0	0	1	1	0	1	0	0	0
LW 100011	0	0	1	0	1	1	1	0	0	0
SW 101011	0	1	0	0	0	1	1	0	0	0
J 000010	1	0	0	0	0	0	0	0	0	0
SHL 10001	0	0	0	0	1	0	0	1	0	0
CMP 101001	0	0	0	1	1	0	0	0	0	1
SUB 100011	0	0	0	1	1	0	0	0	1	0
MOV 111111	0	0	0	0	1	0	0	0	0	0

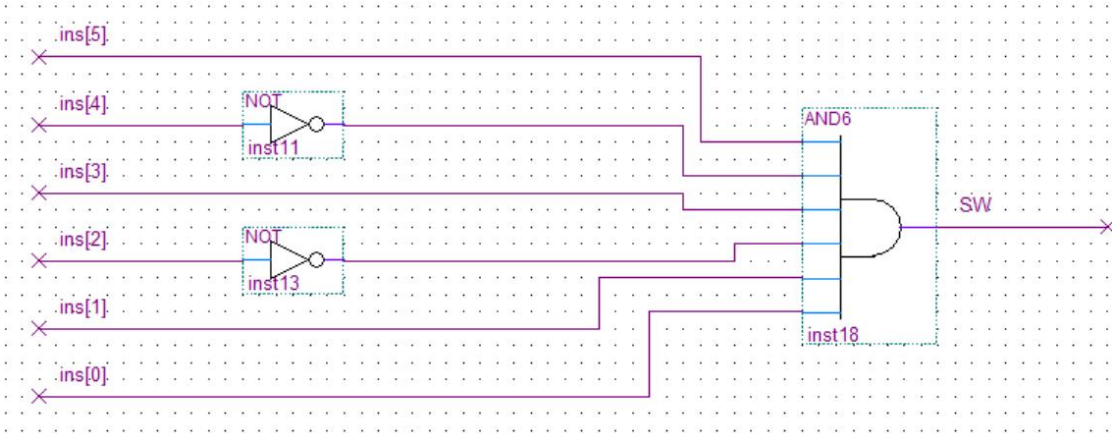
根据控制信号，可以设计出逻辑连线图



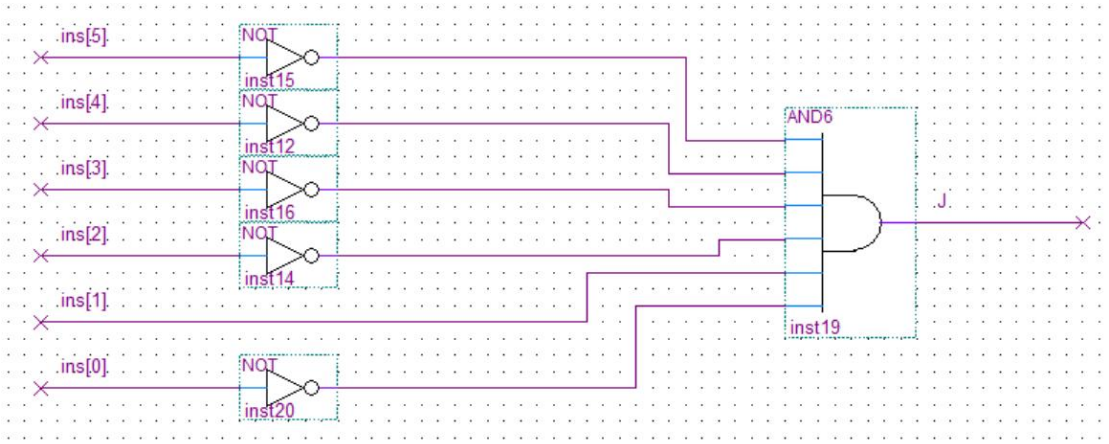
ADD 指令转换图



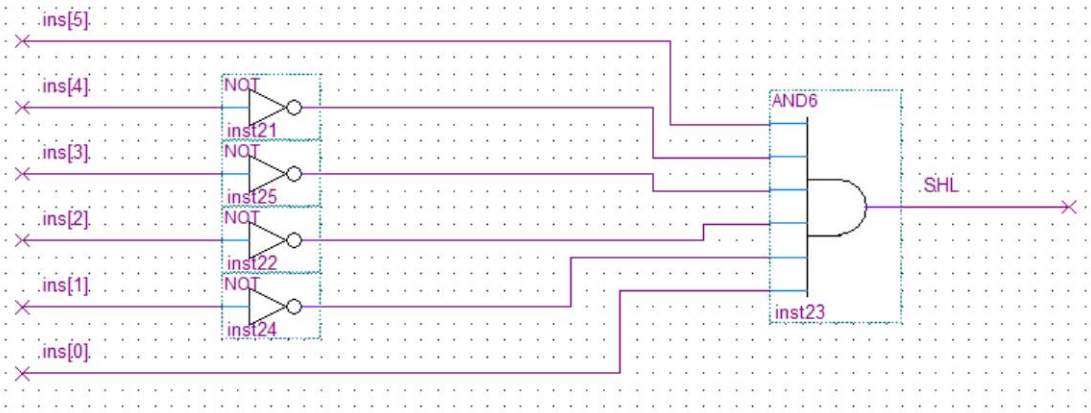
LW 指令转换图



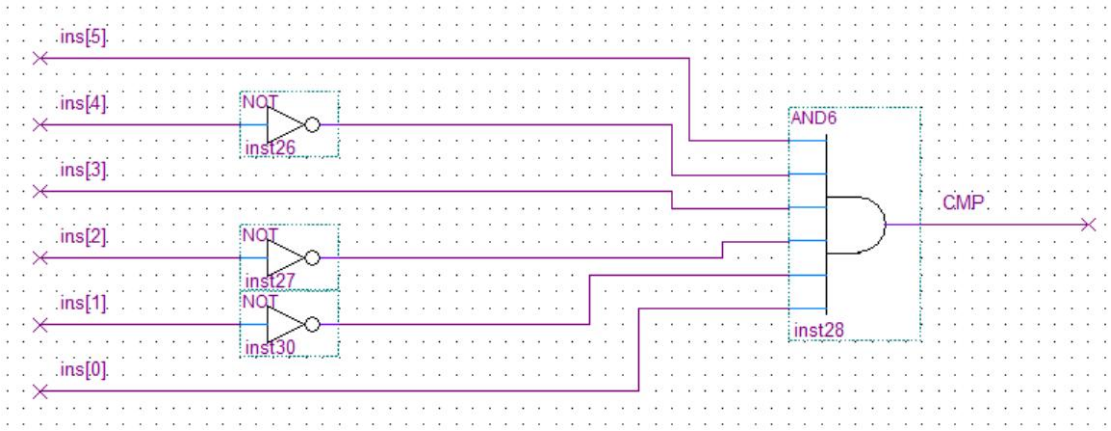
SW 指令转换图



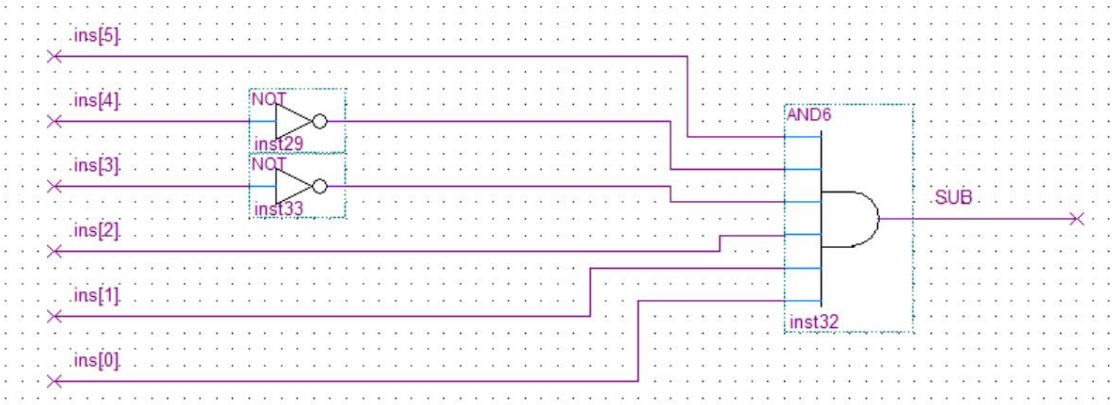
J 指令转换图



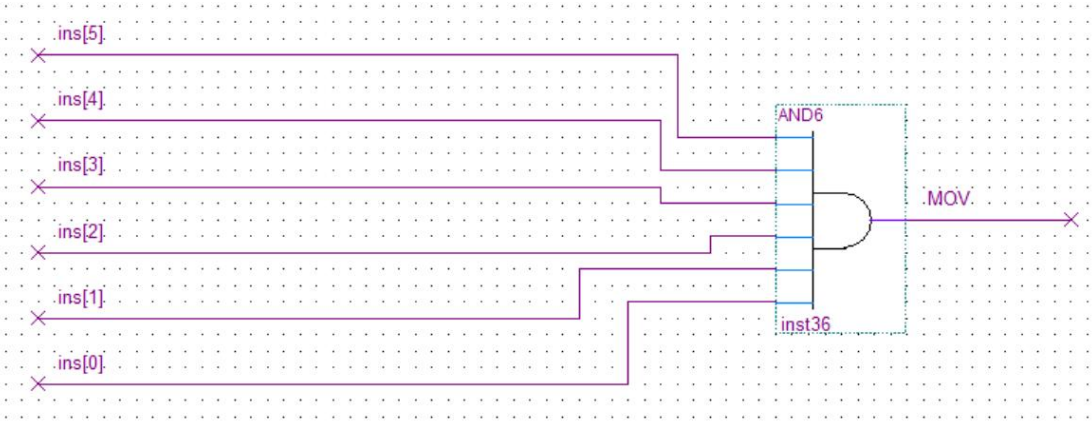
SHL 指令转换图



CMP 指令转换图



SUB 指令转换图



MOV 指令转换图

（1） ADD 指令

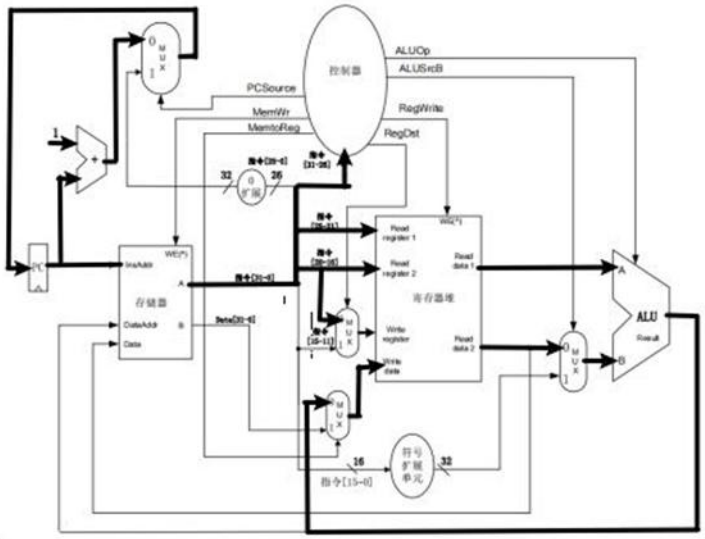


图4.1-1 ADD 指令处理过程图

从图 4.1-1 可以看到，经过一个时钟后 PC 自动完成加 1.。从存储器取出指令，[31..26]

进入控制器译码产生控制信号，[25..21]和[20..16]作为寄存器地址从寄存器堆中取出两加数 到 ALU,ALU 运算结果在时钟下一个上升沿写回寄存器地址[15..11]。因此有效控制信号 RegDst、 ALUOp、RegWrite 均为 1，其余控制信号为 0。

（2） LW 指令

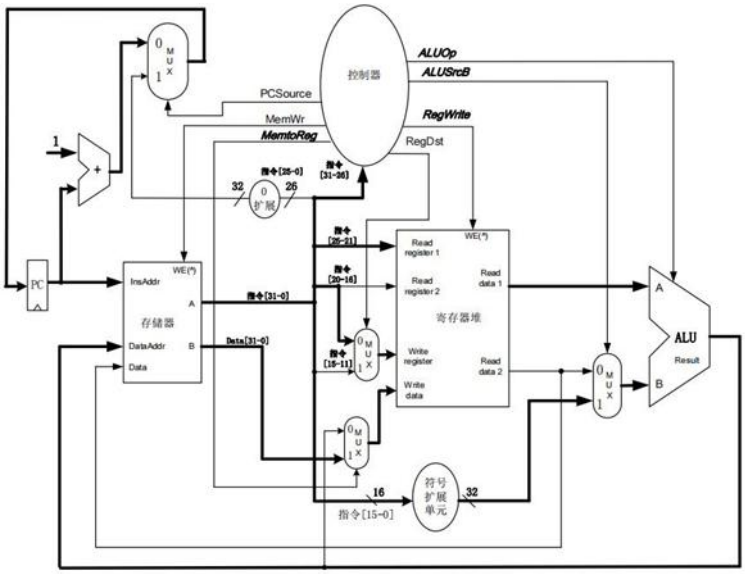


图4.1-2 LW 指令处理过程图

从图 4.1-2 可以看到，指令[31..26]进入控制器译码产生控制信号，[25..21]作为寄存器地址 读出基地址，[15..0]立即数符号扩展，与基地址在 ALU 相加，作为存储器地址。存储器从 B 口读出数据，在时钟下一个上升沿写到寄存器堆地址[20..16]中。因此有效控制信号 ALUSrcB、ALUOp、RegWrite、MemtoReg 均为 1，其余控制信号为 0。

（3） SW 指令

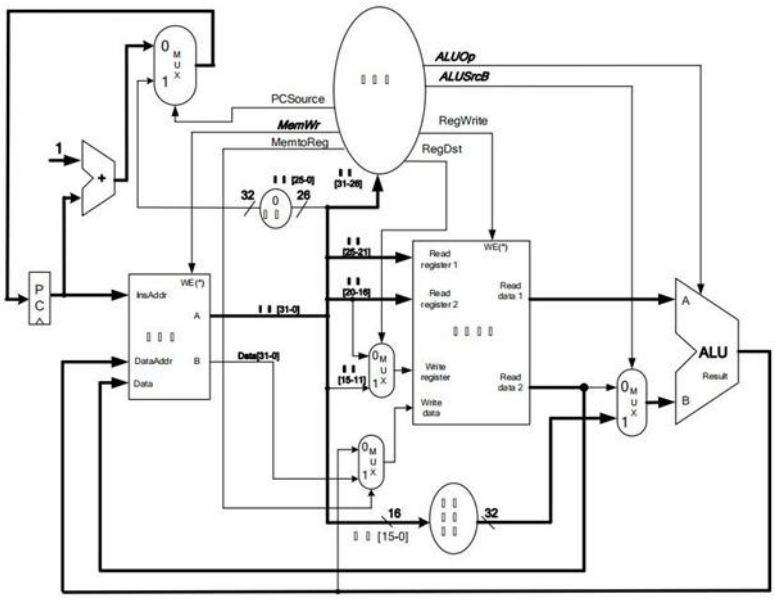


图4.1-3 SW 指令处理过程图

从图 4.1-3 可以看到， 指令[31..26]进入控制器译码产生控制信号，[25..21]作为寄存器地址读出基地址，[15..0]立即数符号扩展，与基地址在 ALU 相加，作为存储器地址。寄存器堆地址[20..16]读出数据到存储器的 Data 端，在时钟下一个上升沿保存。因此有效控制信号 ALUSrcB、ALUOp、MemWr 均为 1，其余控制信号为 0。

（4） J 指令

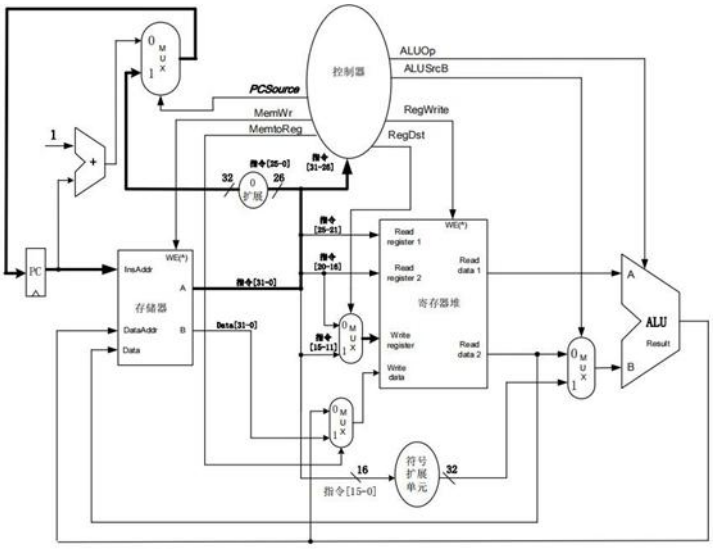


图4.1-4 J 指令处理过程图

从图 4.1-4 中可以看出，指令[31..26]进入控制器译码产生控制信号，立即数[25..0]扩展为 32 位，在时钟下一个上升沿保存到 PC，完成跳转。因此有效控制信号 PCSource 为 1，其余控制信号为 0。

（5） SHL 指令

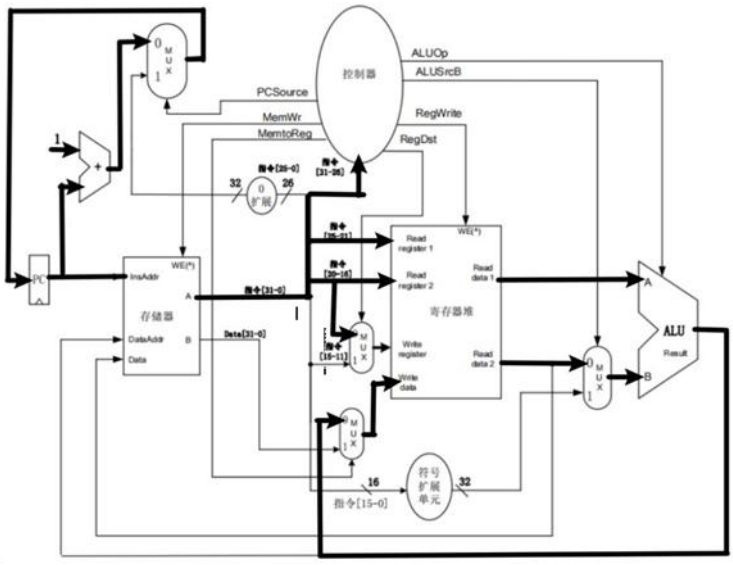


图4.1-5 SHL 指令处理过程图

从图 4.1-5 可以看到，经过一个时钟后 PC 自动完成加 1.。从存储器取出指令，[31..26]

进入控制器译码产生控制信号，[25..21]和[20..16]作为寄存器地址从寄存器堆中取出两加数 到 ALU,ALU 运算结果在时钟下一个上升沿写回寄存器地址[15..11]。因此有效控制信号 ALUOp、 RegWrite 均为 1，其余控制信号为 0。

（6） CMP 指令

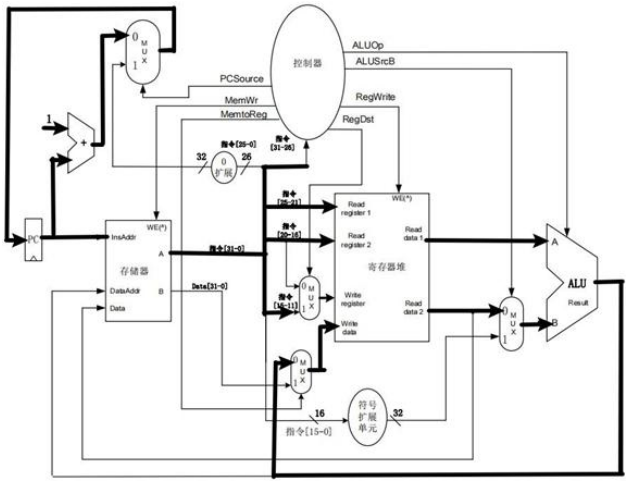


图4.1-6 CMP 指令处理过程图

从图 4.1-6 可以看到，经过一个时钟后 PC 自动完成加 1.。从存储器取出指令，[31..26]

进入控制器译码产生控制信号，[25..21]和[20..16]作为寄存器地址从寄存器堆中取出两加数 到 ALU,ALU 运算结果在时钟下一个上升沿写回寄存器地址[15..11]。因此有效控制信号 ALUOp、 RegWrite 均为 1，其余控制信号为 0。

（7） SUB 指令

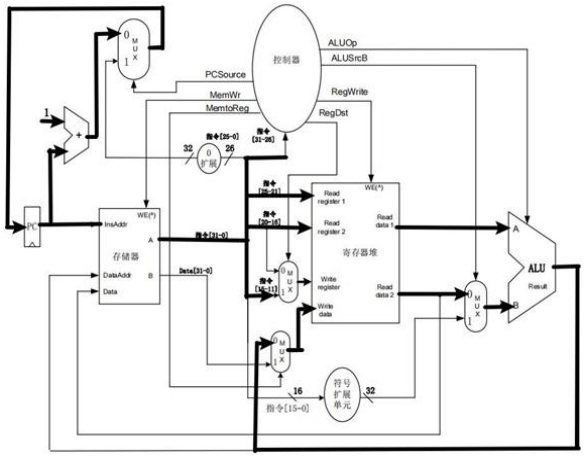


图4.1-7 SUB 指令处理过程图

从图 4.1-7 可以看到，经过一个时钟后 PC 自动完成加 1.。从存储器取出指令，[31..26]

进入控制器译码产生控制信号，[25..21]和[20..16]作为寄存器地址从寄存器堆中取出两加数 到 ALU,ALU 运算结果在时钟下一个上升沿写回寄存器地址[15..11]。因此有效控制信号 RegDst、 ALUOp、RegWrite 均为 1，其余控制信号为 0。

4.2 器件单元模块

1.PC 程序计数器模块

设计思路：

因为 32 位寄存器的初始值是未知的，为了使程序运行在控制状态，我们需要初始化 PC 值，由此我们设计了 set0 模块，其在初始时为 0，而后不段输出 1，该器件与 MUX 配合实现 PC 的初始化功能。

连接线如下所示：

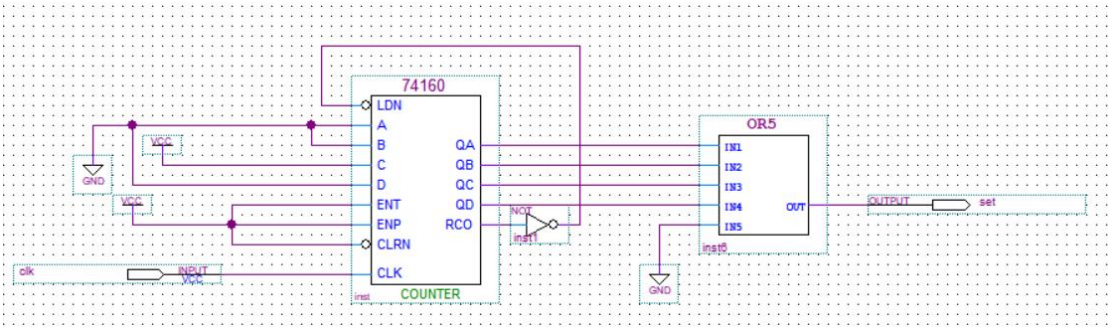


图.set0 模块线路图

封装后如图：

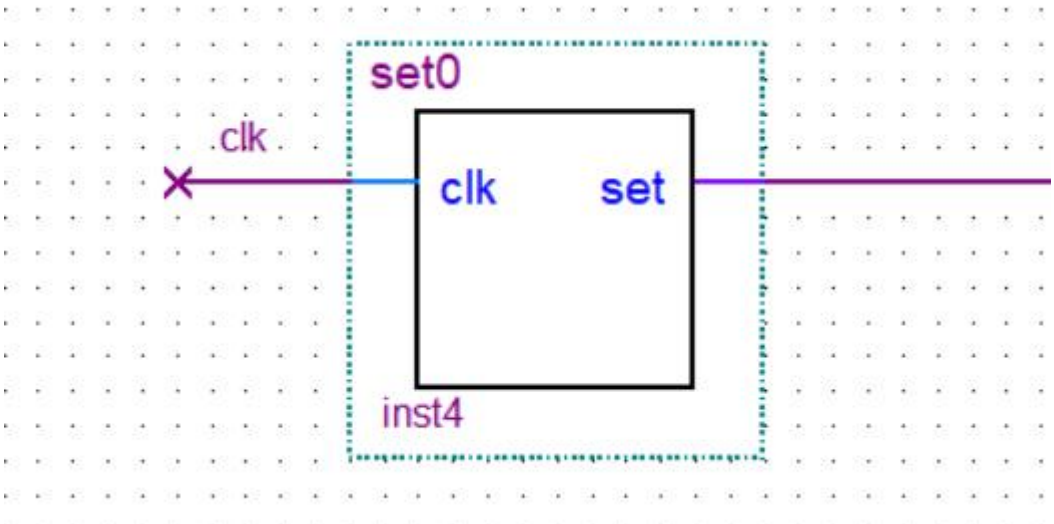
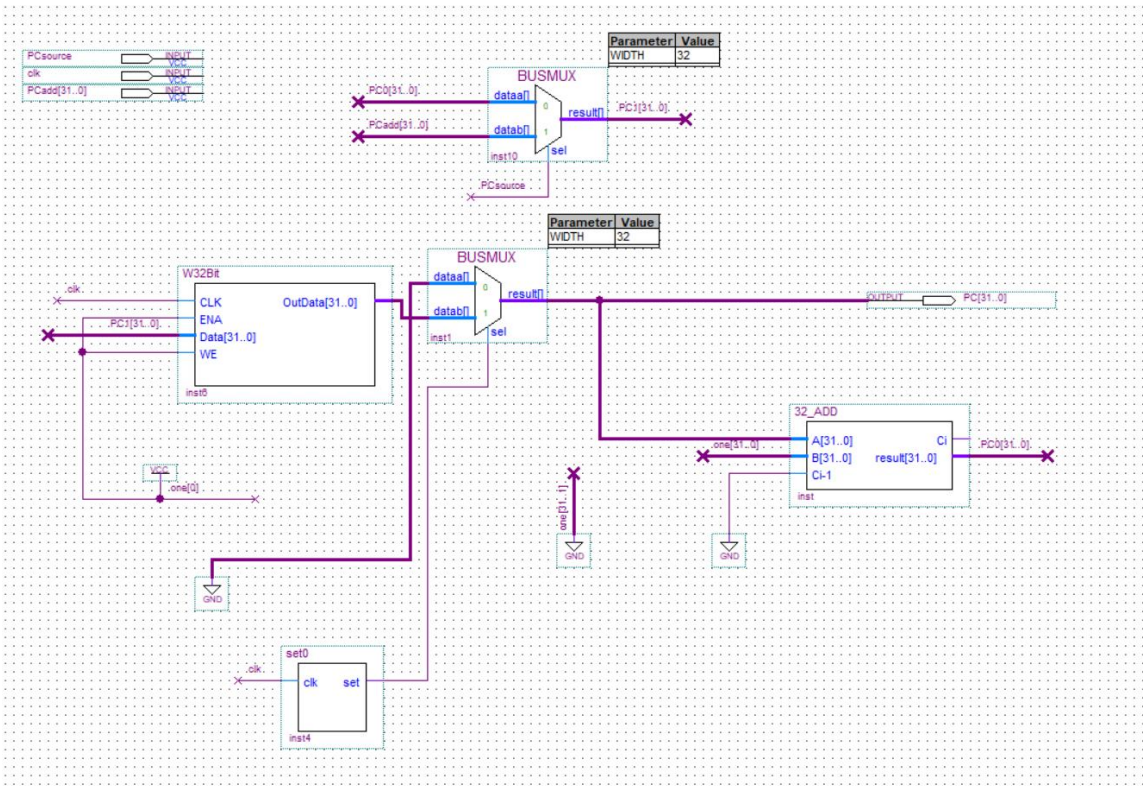


图.set0 模块封装图

为了实现指令的存取和地址的自动+1，PC 至少要有两部分来组成，一个 32 位的寄存器和一个 32 位的 ALU 加法器，加法器+1 后的运算结果不仅要输出，还要重新输入到寄存器中以完成顺序执行指令。同时，为了实现 J 指令，还需要一个数据选择器，来判断是否需要寄存器内的指令覆盖为需要跳转的指令。

连接线如下所示：



图：PC 模块内部线路图

封装后如图：

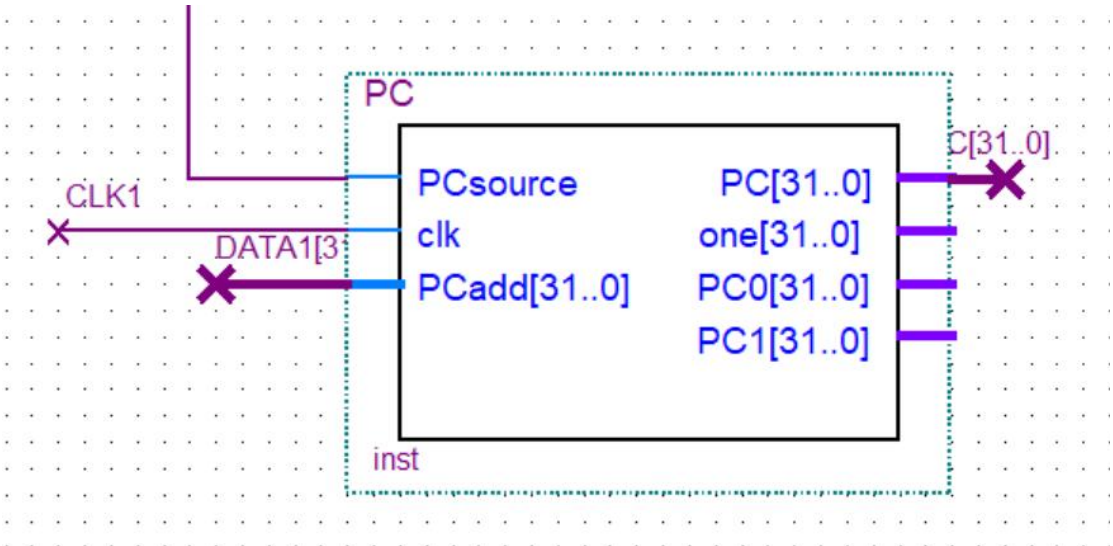


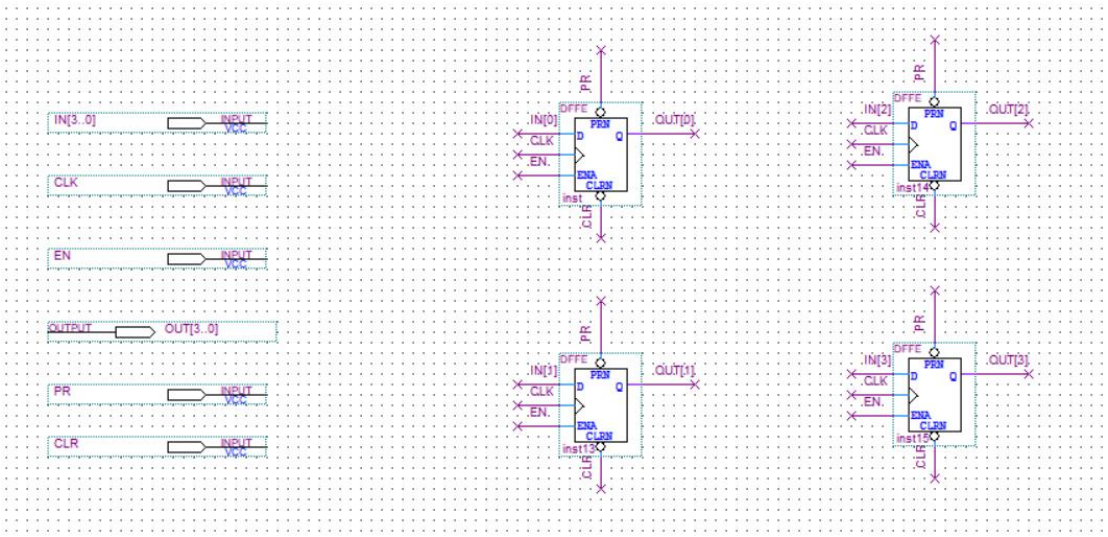
图.PC 模块封装图

2.寄存器堆

设计思路：

为得到一个 32 位的寄存器堆，我们需要 32*32 位寄存器，而 32 位寄存器可通过 1 位的 D 触发器组合而成。我们通过 4 个 D 触发器组合成 4 位寄存器，扩展为 16 位触发器，由 16 位触发器扩展成 32 位寄存器。通过 5_32 译码器实现地址选择，完成片选、读写信号选择。5_32 译码器通过四个 74138 芯片拓展实现。最终 32*32 数据选择器来实现输出。将上述组件进行逻辑组合连线，完成寄存器堆的设计。

(一) 4 位寄存器：使用 4 个 D 触发器连接组成，连接方式如下图



图：4 位寄存器线路图

封装后如图所示：

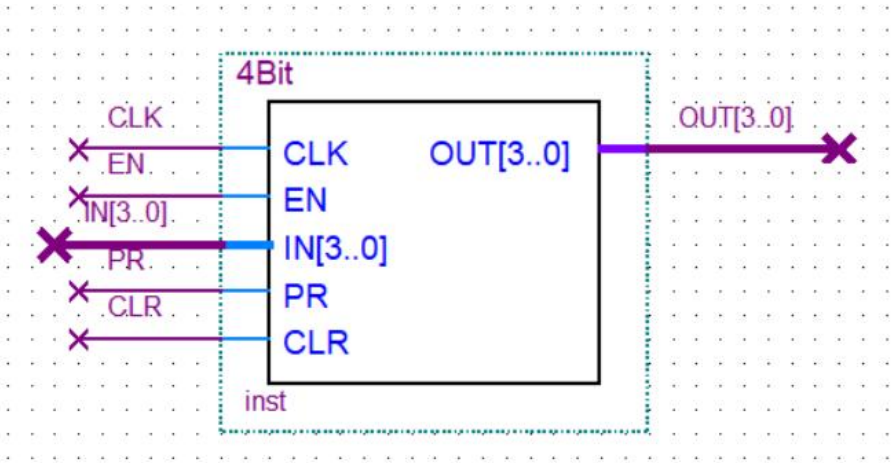


图.4 位寄存器封装图

(二) 8 位寄存器：使用 2 个四位寄存器连接而成

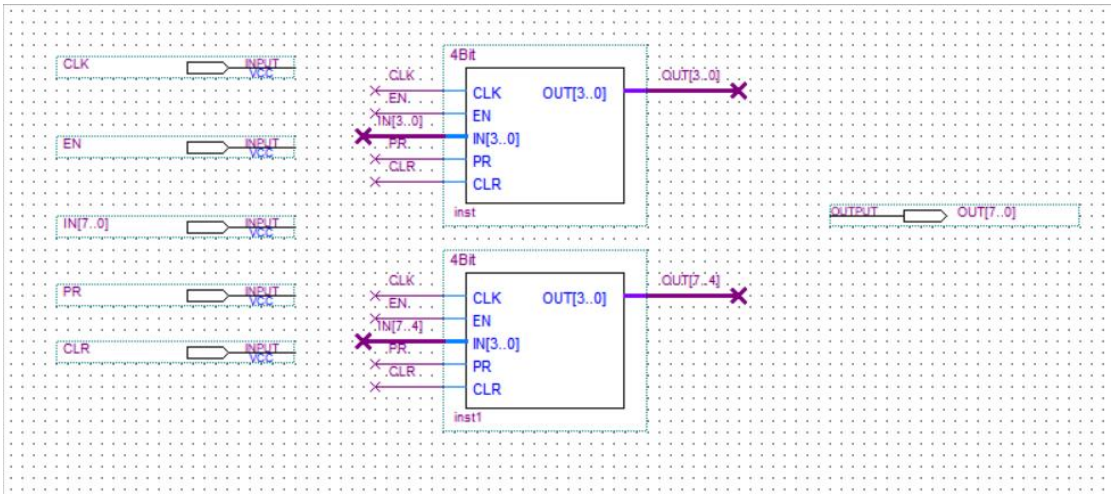
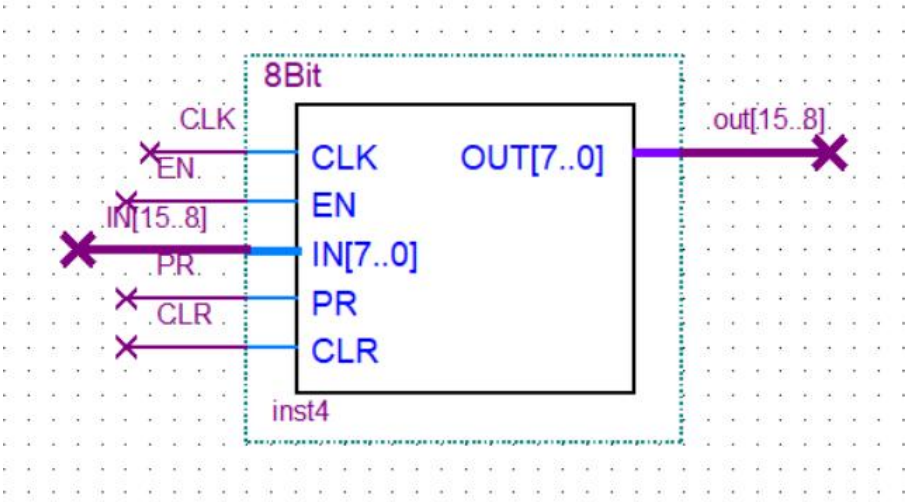


图.8 位寄存器线路图

封装后如图所示：



(三) 32 位寄存器：使用 4 个八位寄存器连接而成：

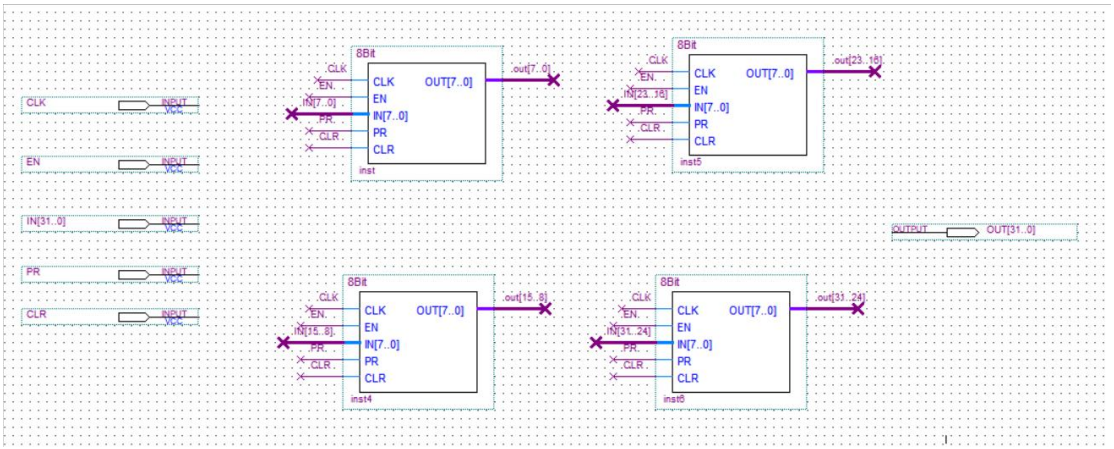
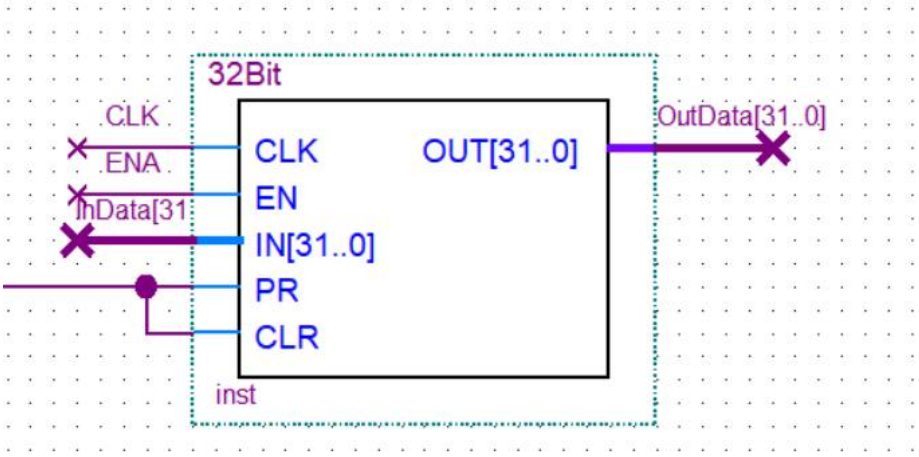


图.32 位寄存器线路图

32 位寄存器封装图：



(四) 带写信好的 32 位寄存器：给之前的 32 位的寄存器添加一个控制写信号 WE

设计思路：

采用一个数据选择器，将 WE 控制信号接到接到数据选择器的选通端，数据选择器的“1”端接要写入的数据，“0”端接寄存器的输出端，再将选择器的输出端接到寄存器的输入端，以此形成一个回流。当 WE 置“0”时，寄存器里面的数据不会更改，当 WE 置“1”时，输入的数据进入寄存器，从而实现写信号功能。

连线如图所示：

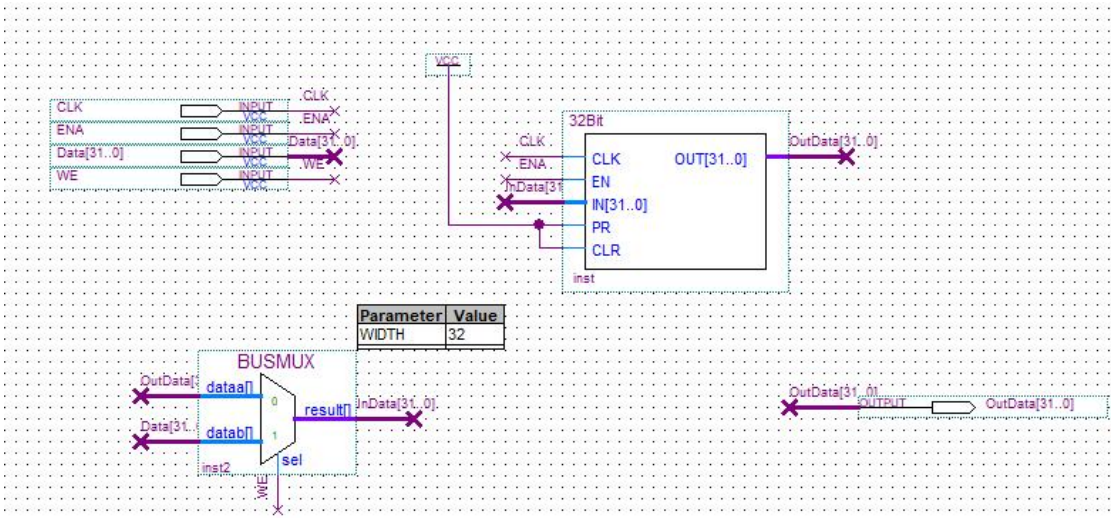
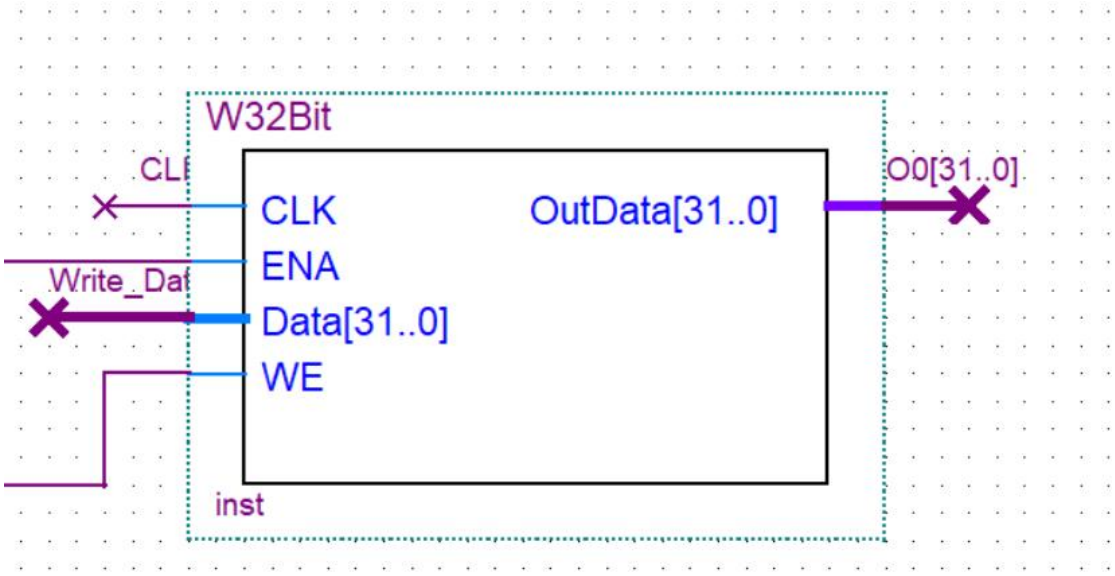


图.带写信号的 32 位寄存器封装图

封装后如图所示：

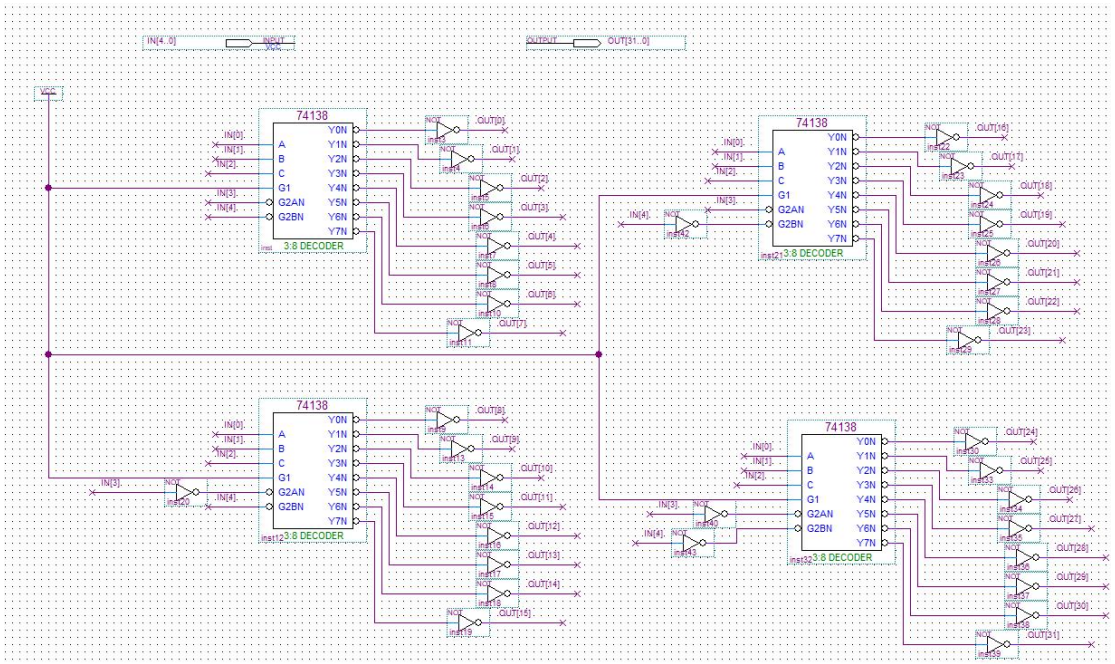


(五) 5_32 译码器

设计思路：

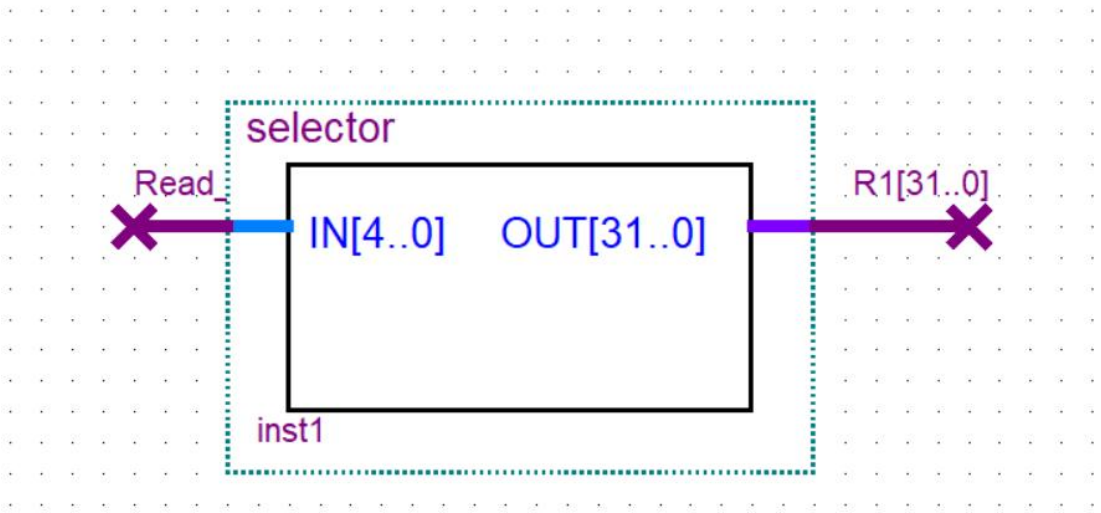
采用 4 个 74138（3-8 译码器），低三位接四个译码器的译码控制端，高 2 位接译码器的使能控制端实现片选功能。

连线如图所示：



5_32 译码器

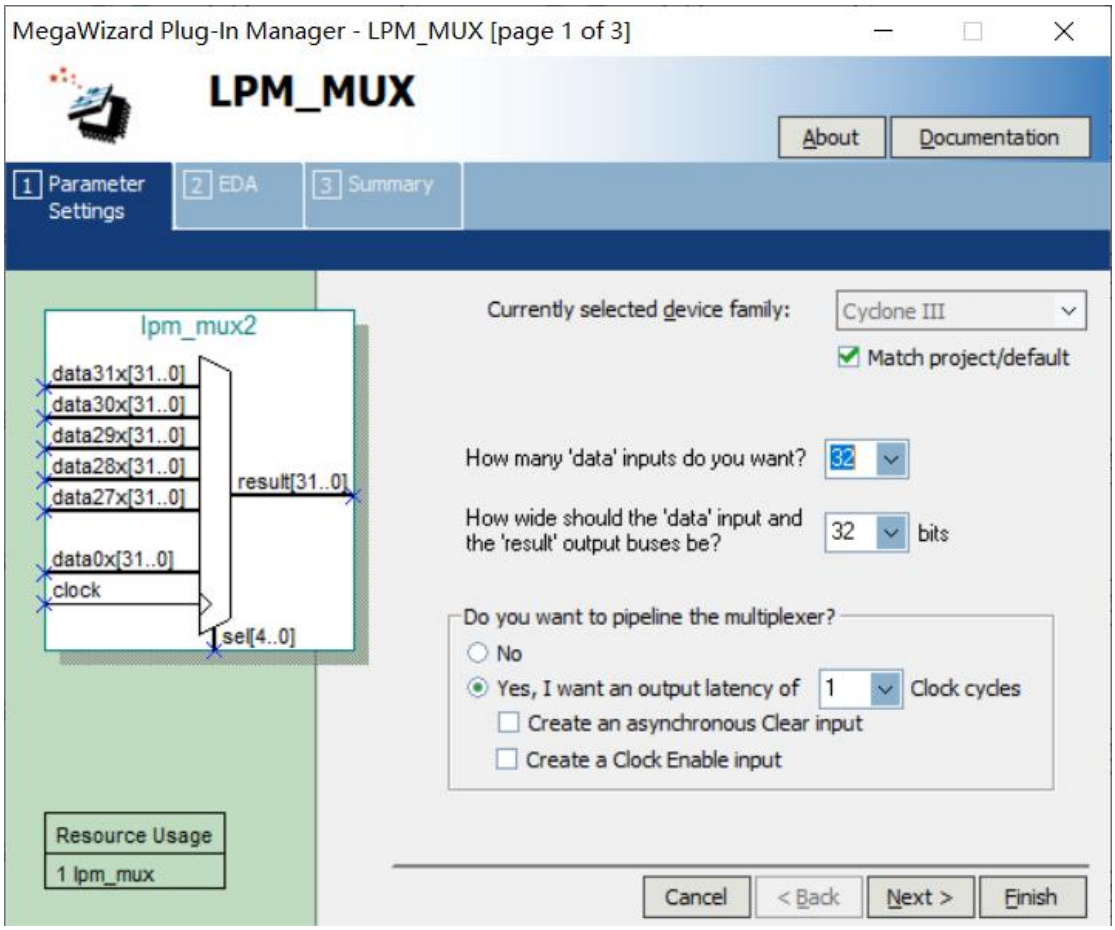
封装后如图所示：



(六) 32*32 数据选择器

设计思路：

采用 LPM_MUX 设计，选择数据的个数设置为 32，每个数据的宽度设置为 32 位。



(七) 寄存器堆

采用 3 个已经设计好的 5_32 译码器，32 个设计好的带写信号控制的 32 位寄存器，2 个设计好的 32*32 的数据选择器。

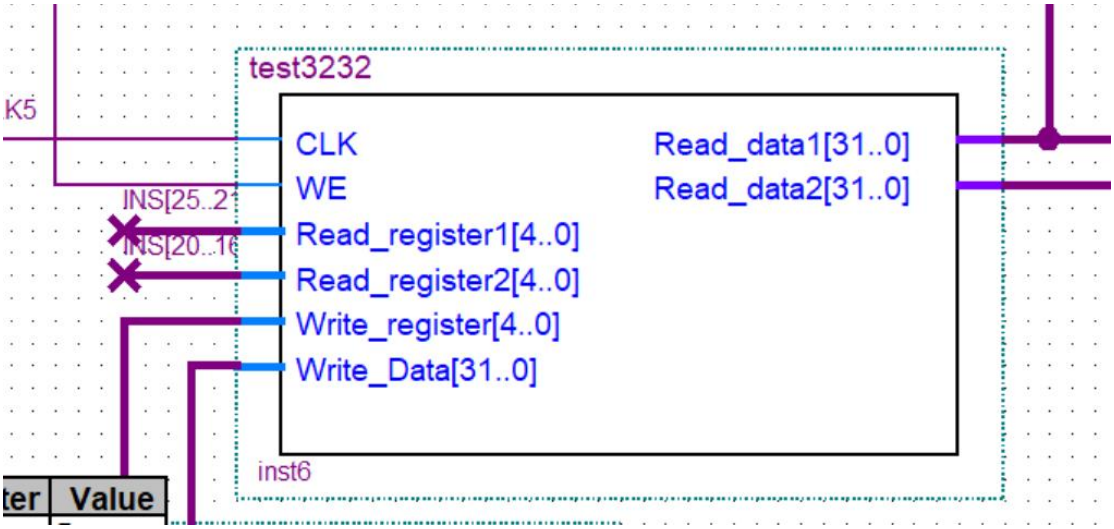
输入寄存器堆的 2 个读地址和 1 个写地址分别接到 3 个 5-32 译码器，而三个 5-32 译码器的 32 个输出端通过 32 个或门接到 32 个寄存器的使能端，这样 3 个译码器中任意一个有数据输出便打开对应寄存器的是能端，实现对 32 位寄存器进行片选的功能。

将写数据地址对应的译码器的输出端和 WE 控制信号通过 32 个与门分别接到 32 个寄存器的 WE 端，这样必须要同时给出 WE 有效信号和写数据地址才能将数据写入寄存器，防止在同时执行读数据和写数据时把数据也一起写道读数据的寄存器中。

写数据并行接入到所有 32 个 32 位寄存器的数据输入端，32 个 32 位寄存器的输出端并行接到 2 个 32*32 数据选择器中。



封装后如图：



3.控制器

设计思路：

控制器可通过指令所给的指令信号输出对应的控制信号，内部通过逻辑连线实现。通过研究设计指令信号，设计对应的逻辑电路以给出对应的控制信号输出，从而完成堆控制器的设计。具体的指令设计在 3-1 中给出。

封装后如图所示：

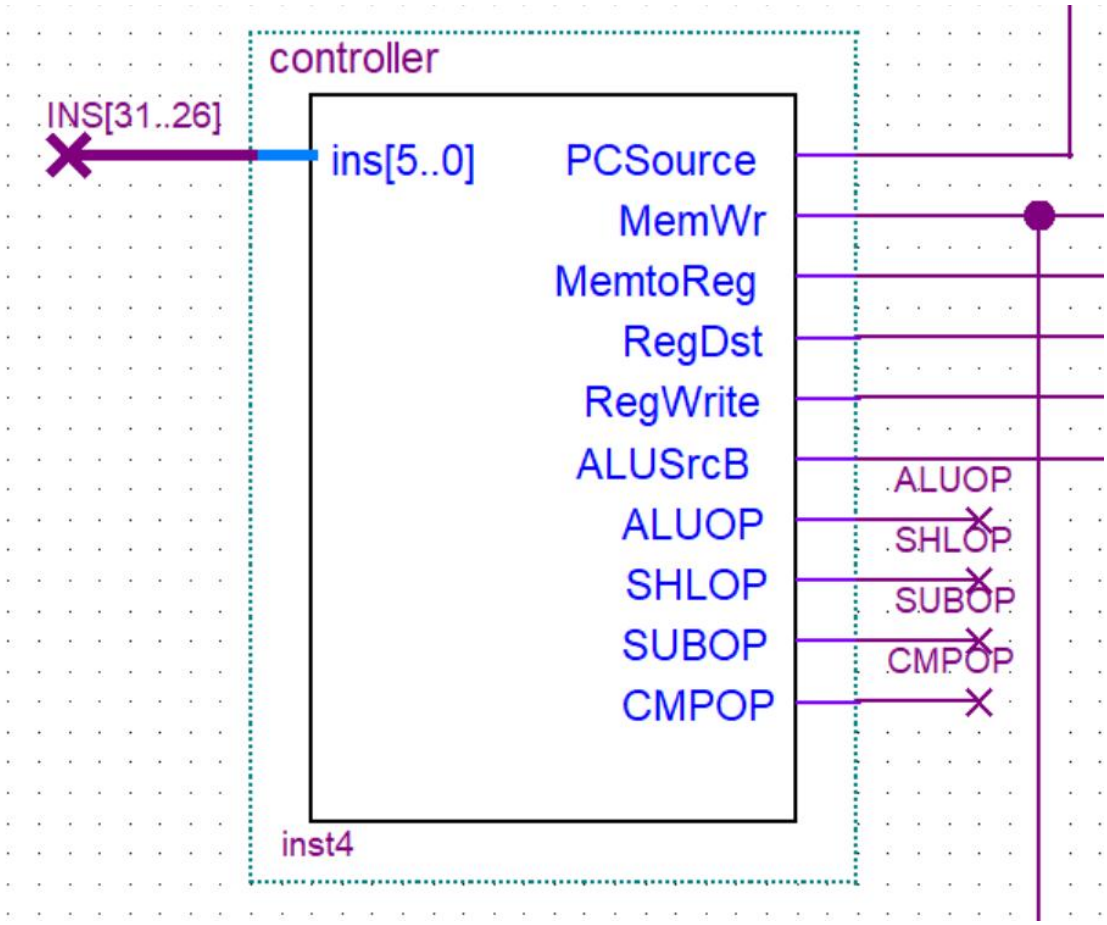


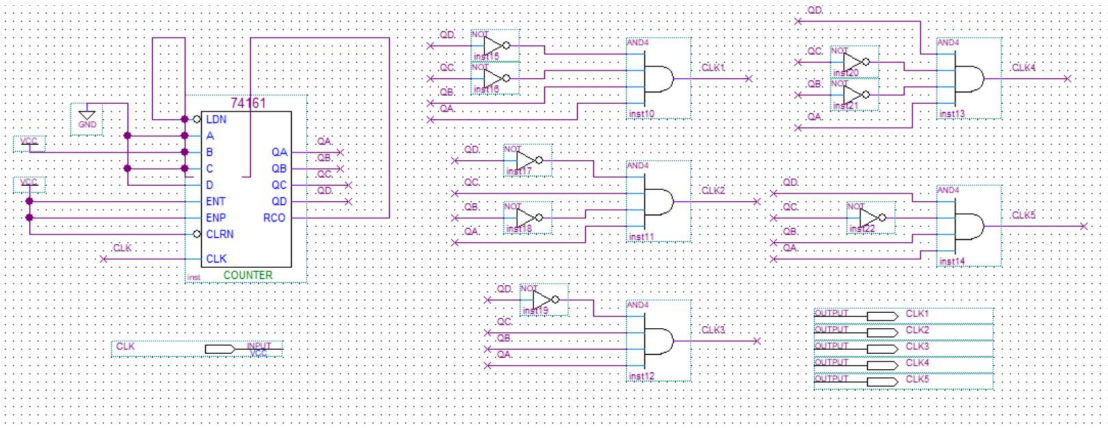
图.控制器模块

4.时钟信号扩展器

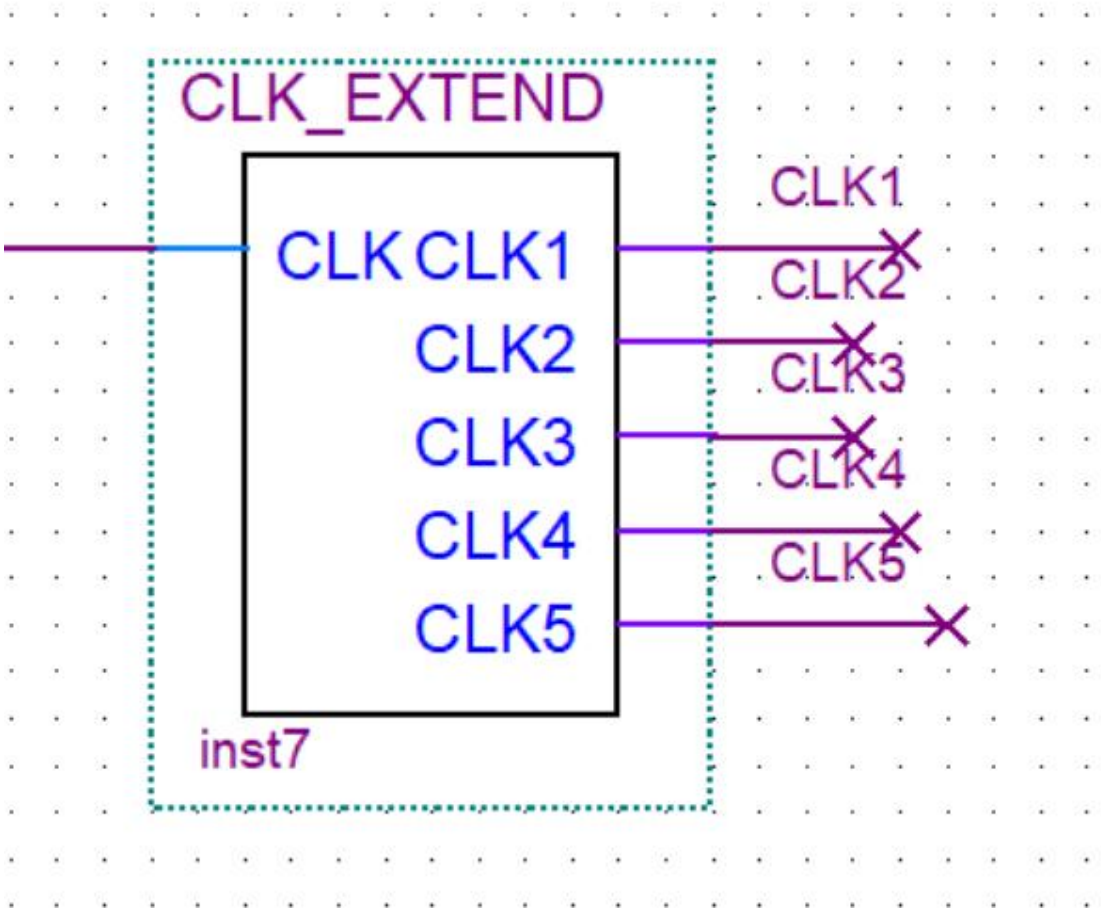
设计思路：

由于各个器件功能不同，其运行的时序也不尽相同，为了满足各器件按照顺序执行，我们设计了一个时钟拓展期间，来将一个时钟信号分为五个时钟信号进行输出。其工作原理为运用 74161 计数器 CLK 接外部 CLK 进位端接 LDN 置数端，实现 2~15 的循环，当计数器计数达到 3 时输出第一个时钟，达到 5 时输出第二个时钟，达到 7 时输出第三个时钟，达到 9 时输出第四个时钟，达到 11 时输出最后一个时钟，从而实现有 1 个时钟信号得到五个分时时钟信号。

其逻辑线路图如下：



封装后如图所示



5. ALU

ADD：ADD 模块底层由半加器组成，半加器通过一个与门和一个或非门实现逻辑。如下图：

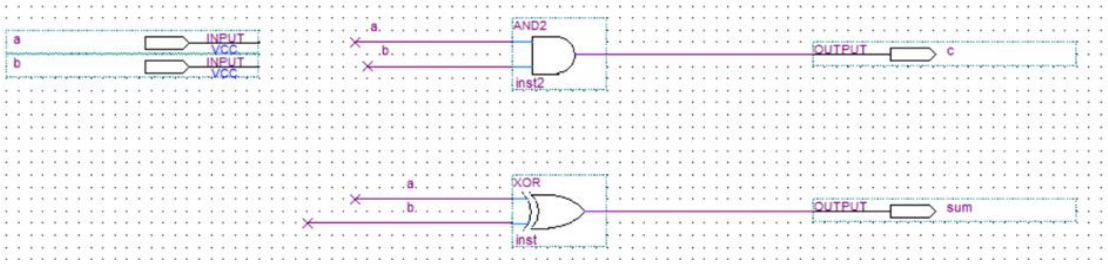


图4.5-1　半加器设计图

封装后如下图：

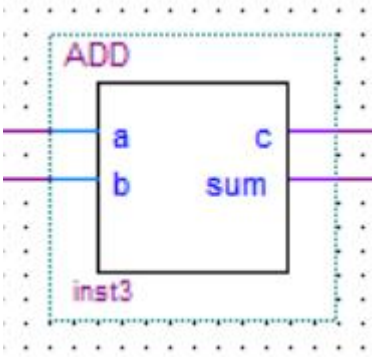


图4.5-2　半加器模块

两个半加器组成一个一位的带进位的全加器。如下图：

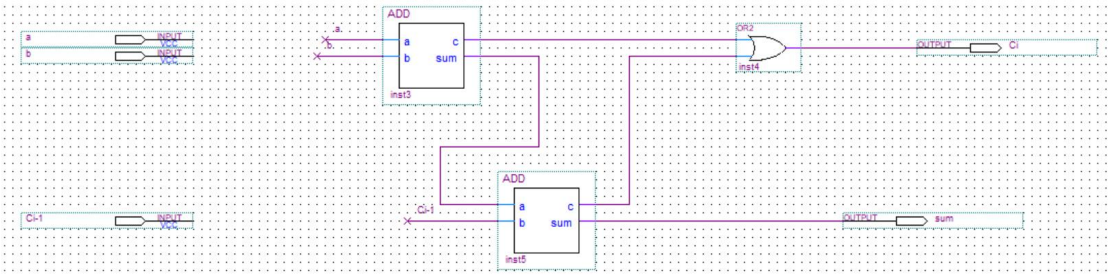


图4.5-3　全加器设计图

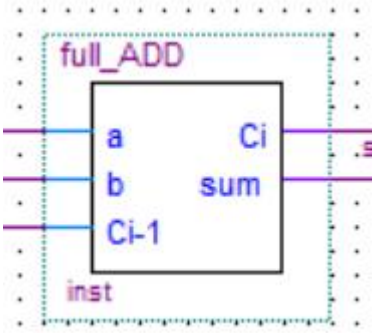


图4.5-4　全加器模块

随后用 8 个全加器组成 8 位全加器，再用 4 个 8 位全加器组成 32 位全加器。

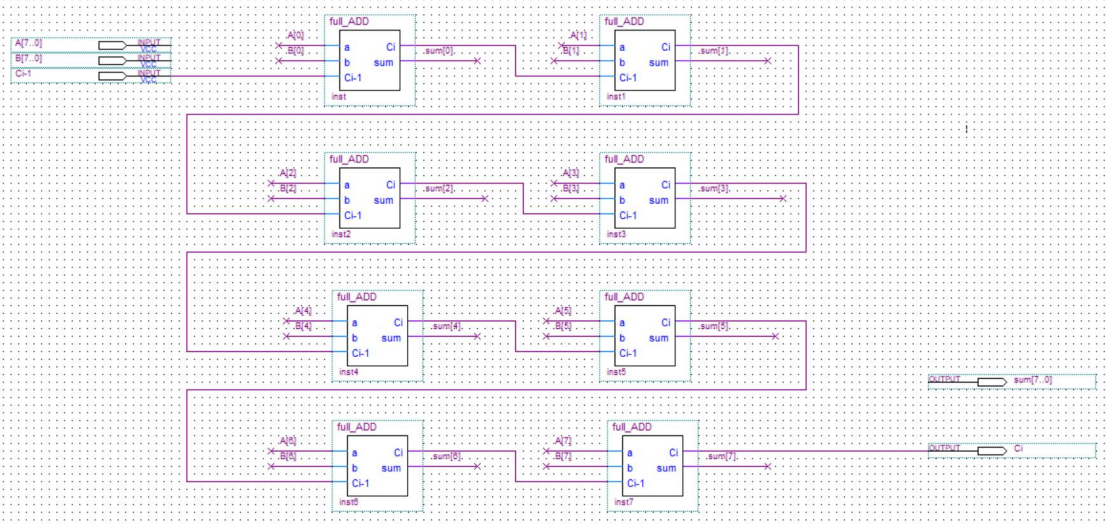


图4.5-5　8 位全加器设计图

32 位全加器如下图：

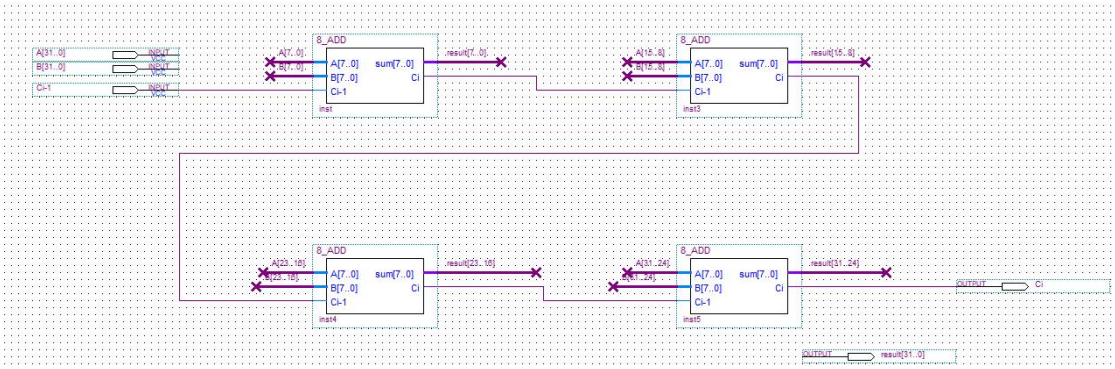
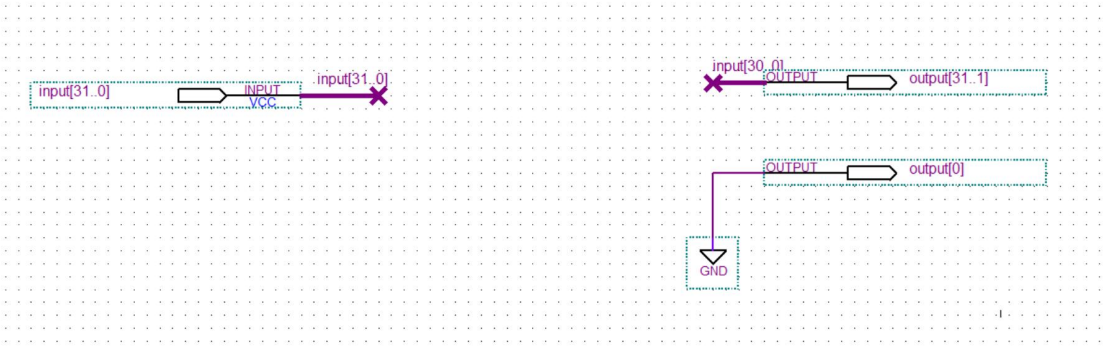
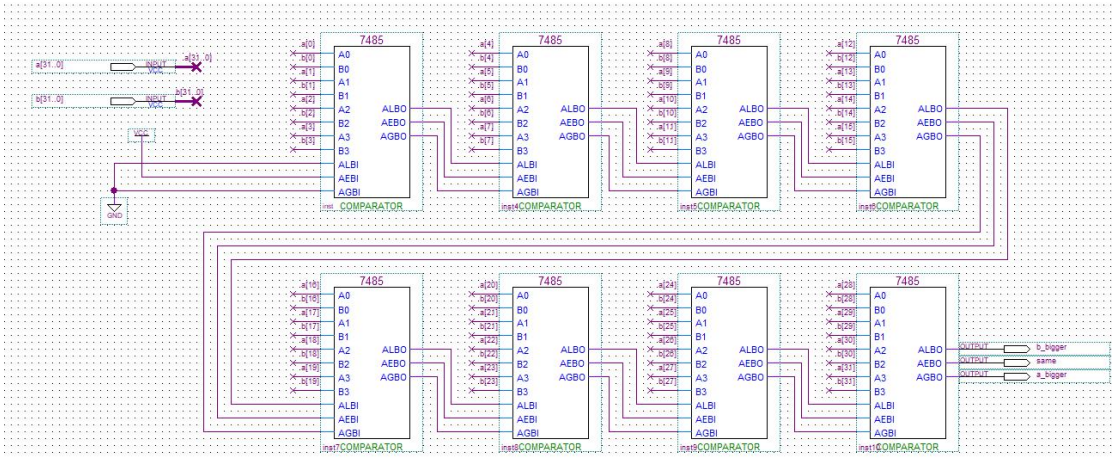


图4.5-6　32 位全加器设计图

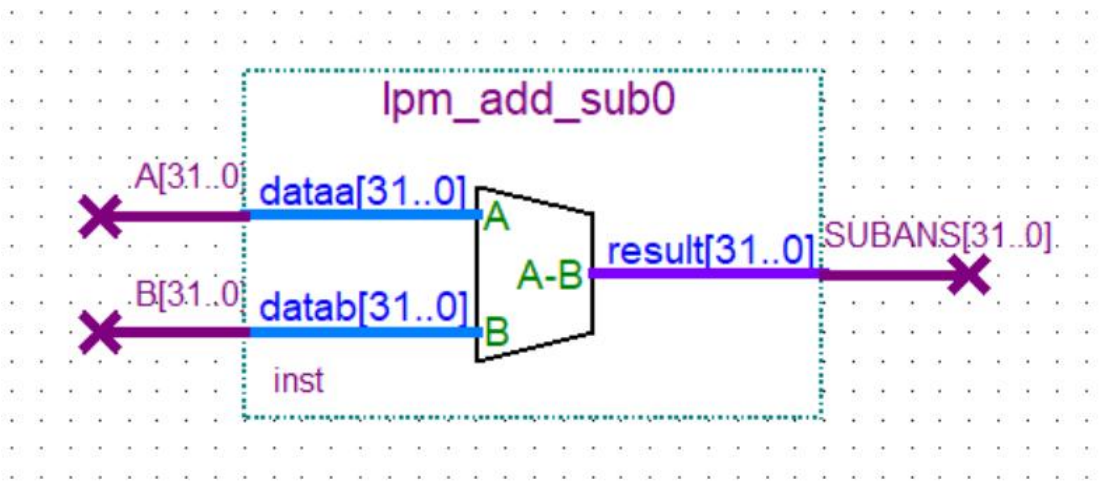
shl:算数左移通过如下电路图实现：



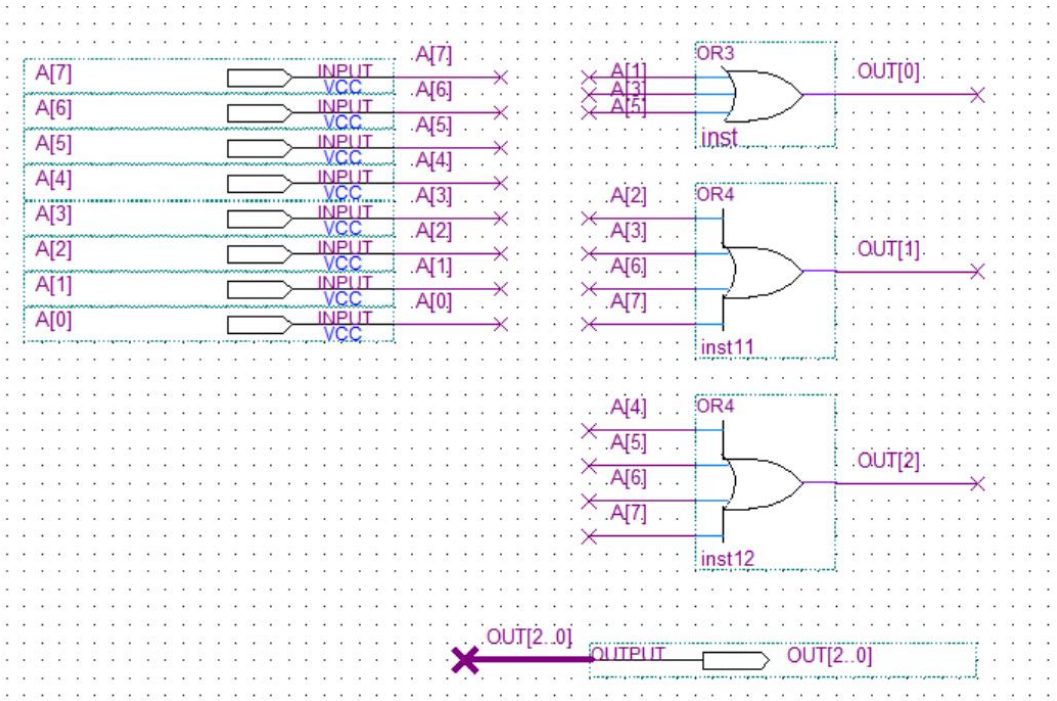
cmp:CMP 模块通过 7438 芯片按位比较，最后输出结果，设计如下图：



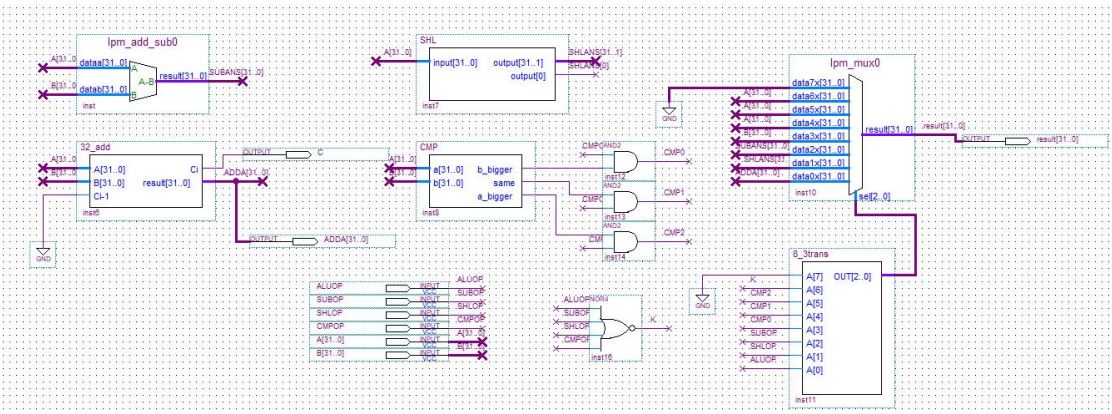
sub：使用内置的 lpm_add_sub 模块生成



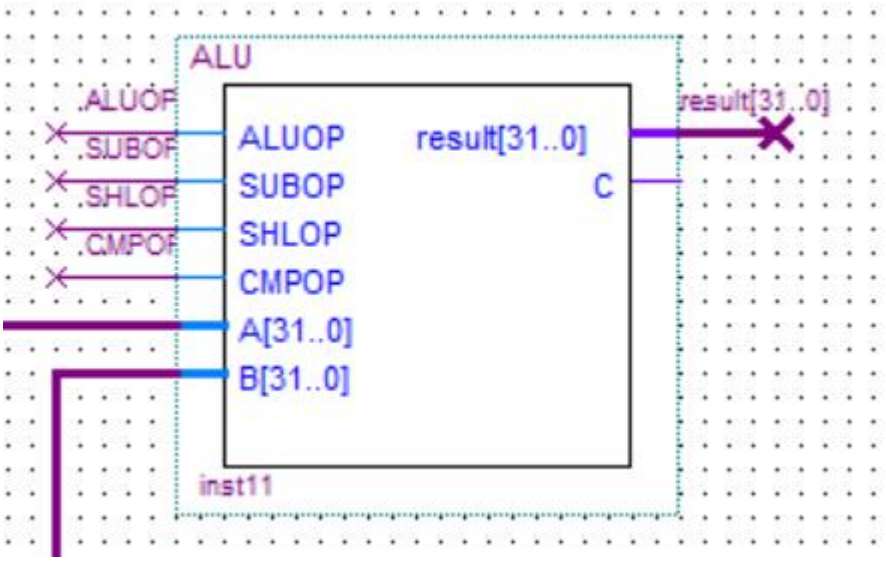
8_3:在数据选择部分需要用到一个 8-3 编码器和一个 8 选 1 的数据选择器。其中 8-3 编码器设计如下图：



8 选1 数据选择器通过 Quartus II 内置模块实现



ALU 最终封装图如下：



6. 存储器

设计思路：存储器模块分为两个部分，一个是 ROM 存储器，用于储存指令；另一个是 RAM 存储器，用于存储数据。我们使用 Quartus II 内部提供的 ROM 和 RAM 组件完成设计。

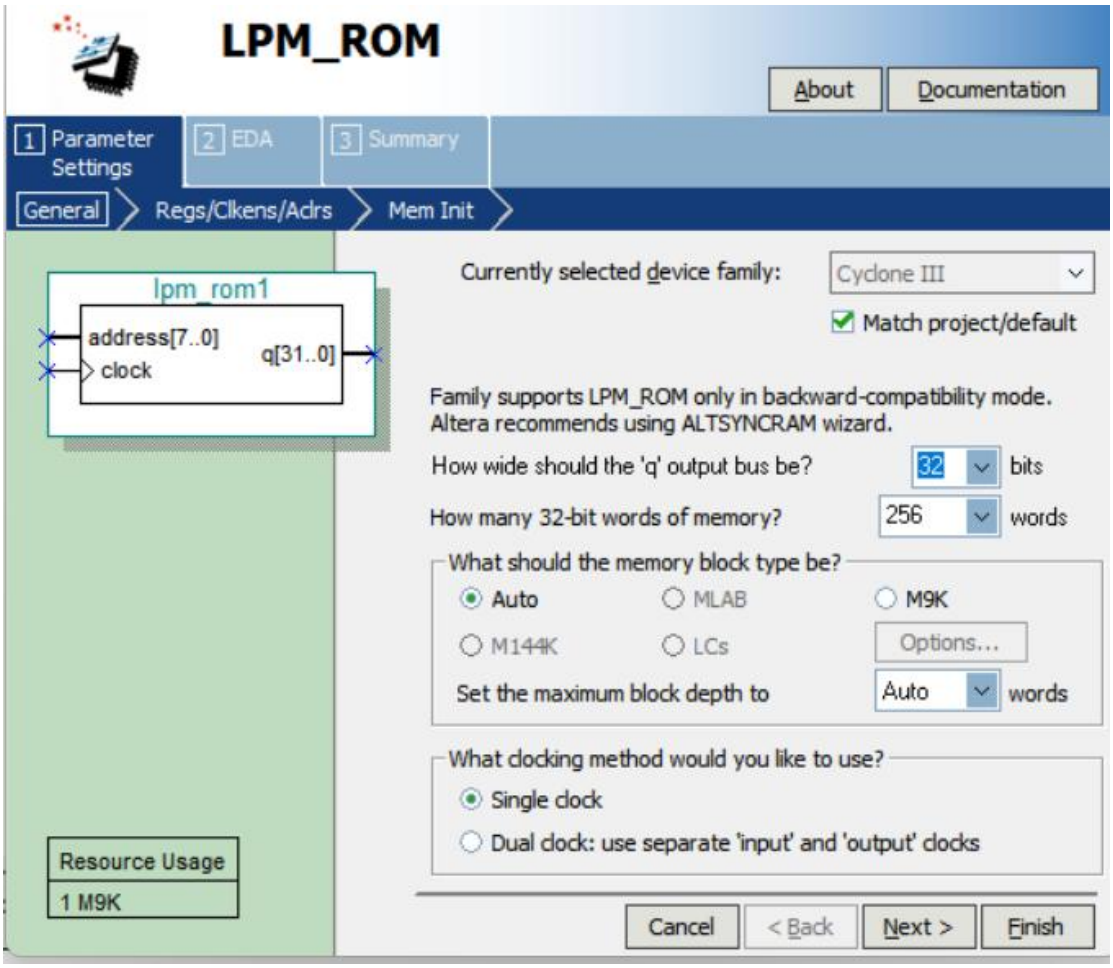
（1） 指令存储器 ROM

指令存储器使用 Quartus II 内部提供的 lpm_rom 来完成设计。首先需要一个.mif 文件完成对存储器的初始化。在 Quartus II 中新建一个.mif 文件，并在指定位置输入设计的指令。如下图：

Addr	+0	+1	+2	+3	+4
0	00000000000000000000000000000000	10001100000000100000010000000	10001100000000100000000010000001	00000000010001000011000000000	10101100000000110000000010000010
8	11111100110001110000000000000000	00001000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000

Addr	+4	+5	+6	+7
0	10101100000000110000000010000010	10000100000100100000000000000000	10100100001000100010100000000000	10011100010000010011000000000000
8	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000

图4.6.1 指令输入



（2） 数据存储器 RAM

数据存储器使用 Quartus II 内部提供的 lpm_ram_dq 来完成设计。同样地，需要一个.mif文件完成存储器的初始化，在 Quartus II 中新建一个.mif 文件，并在地址为 128 和 129 的位置输入数据 1 和 10。

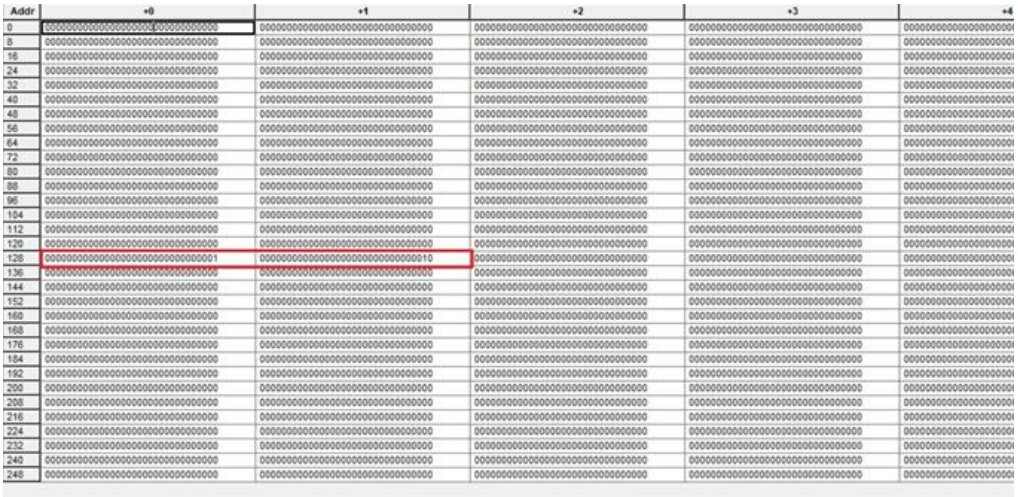
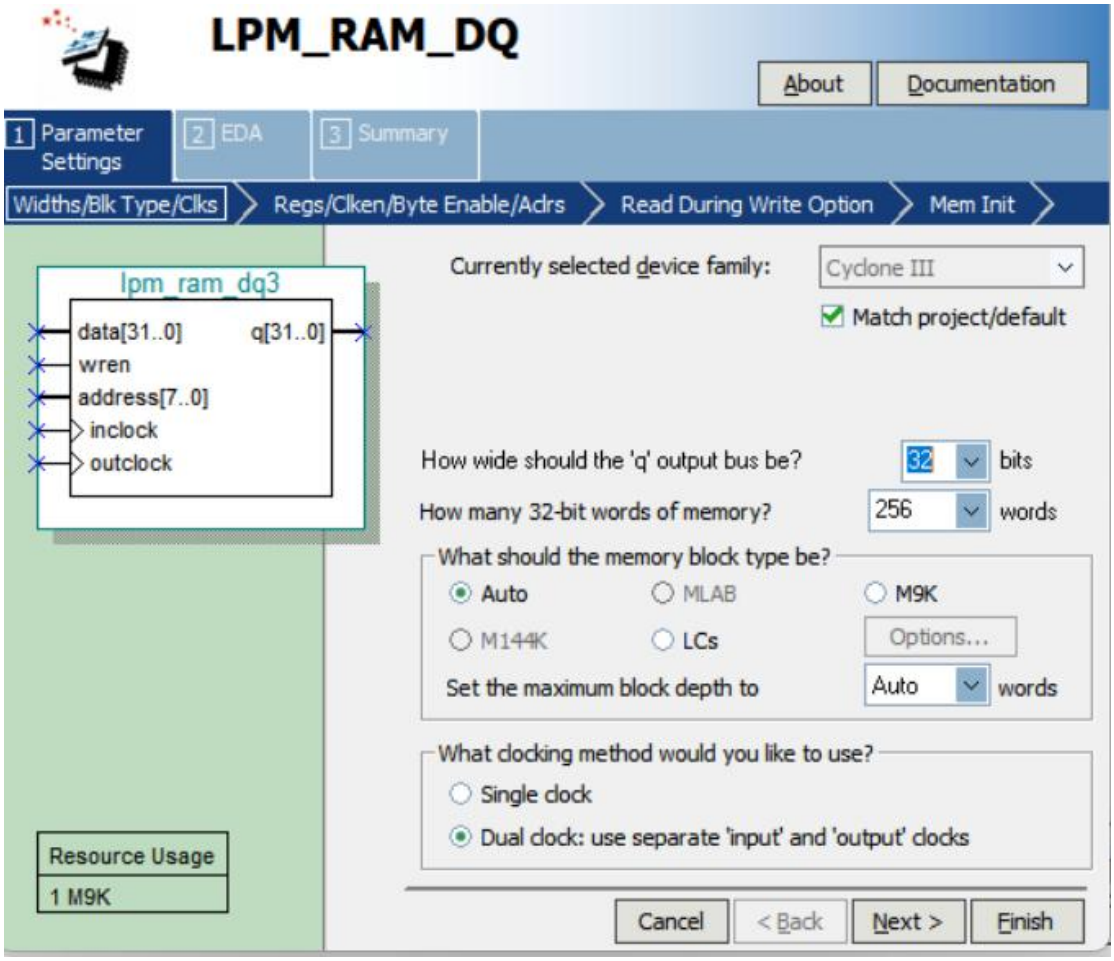
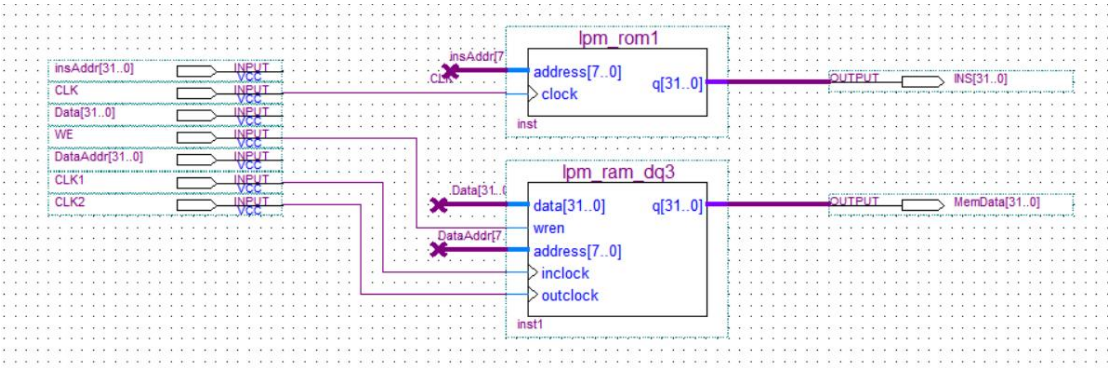


图4.6.2 数据输入

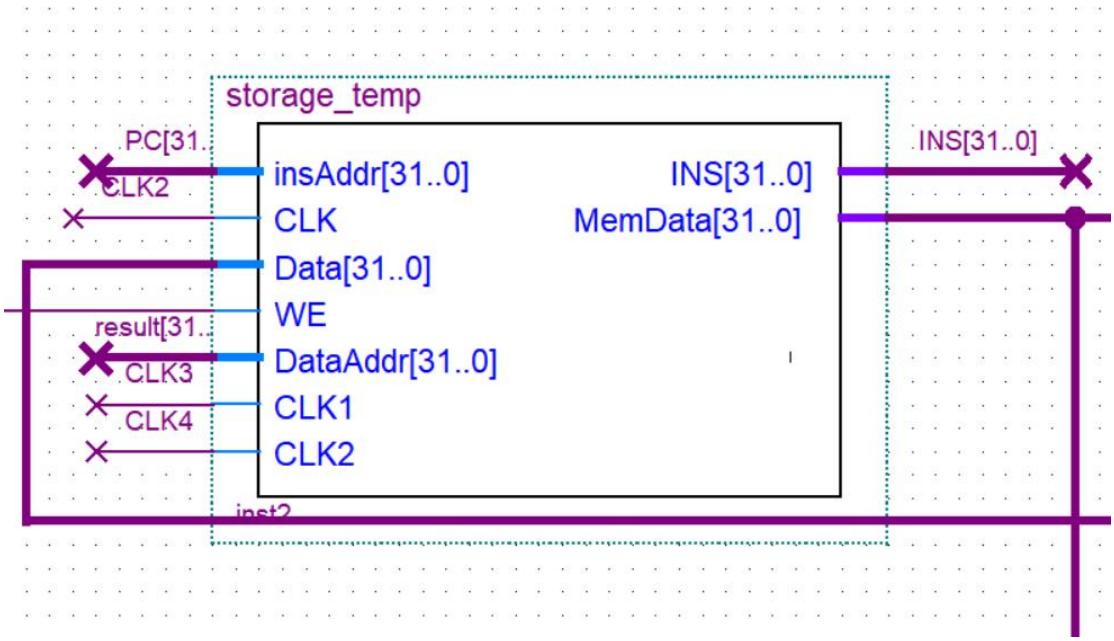


RAM 数据存储器的输入是一个 32 位长度的数据、一位读写控制位、一个 8 位的地址以及两个时钟信号，输出是一个 32 位的数据。

在设计完成 ROM 和 RAM 之后，需要进行组装，完成模块化，如下图：



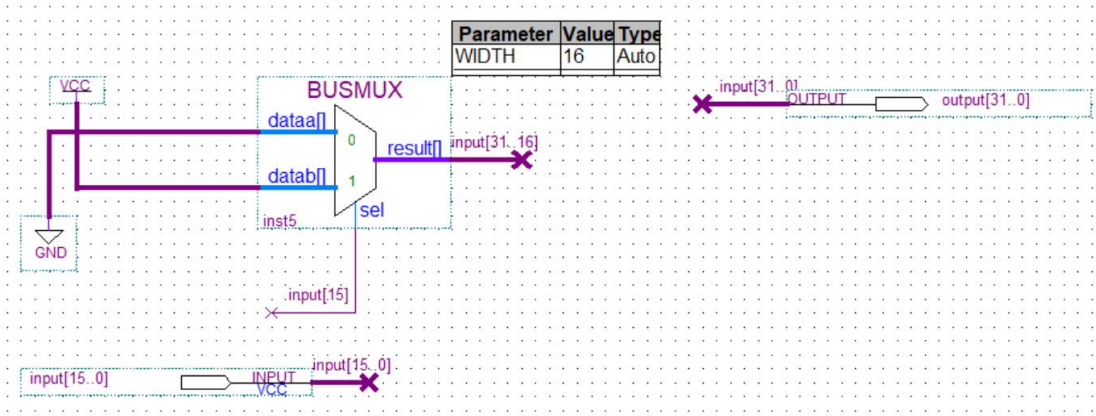
封装图

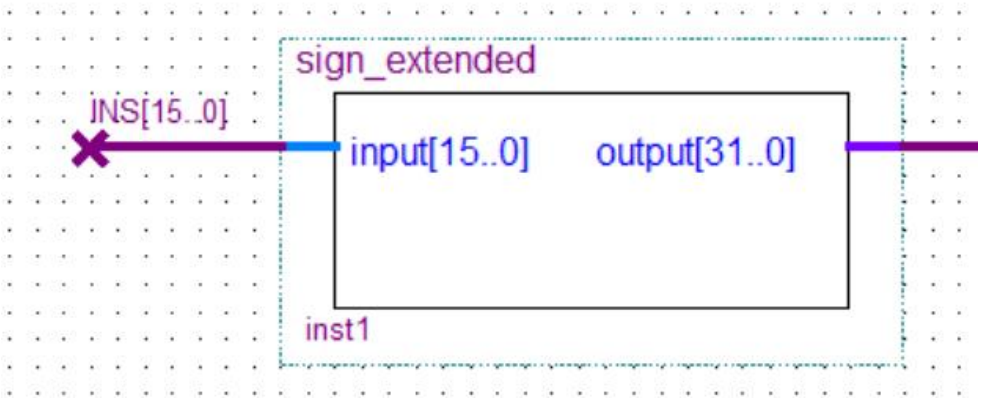


7. 拓展单元

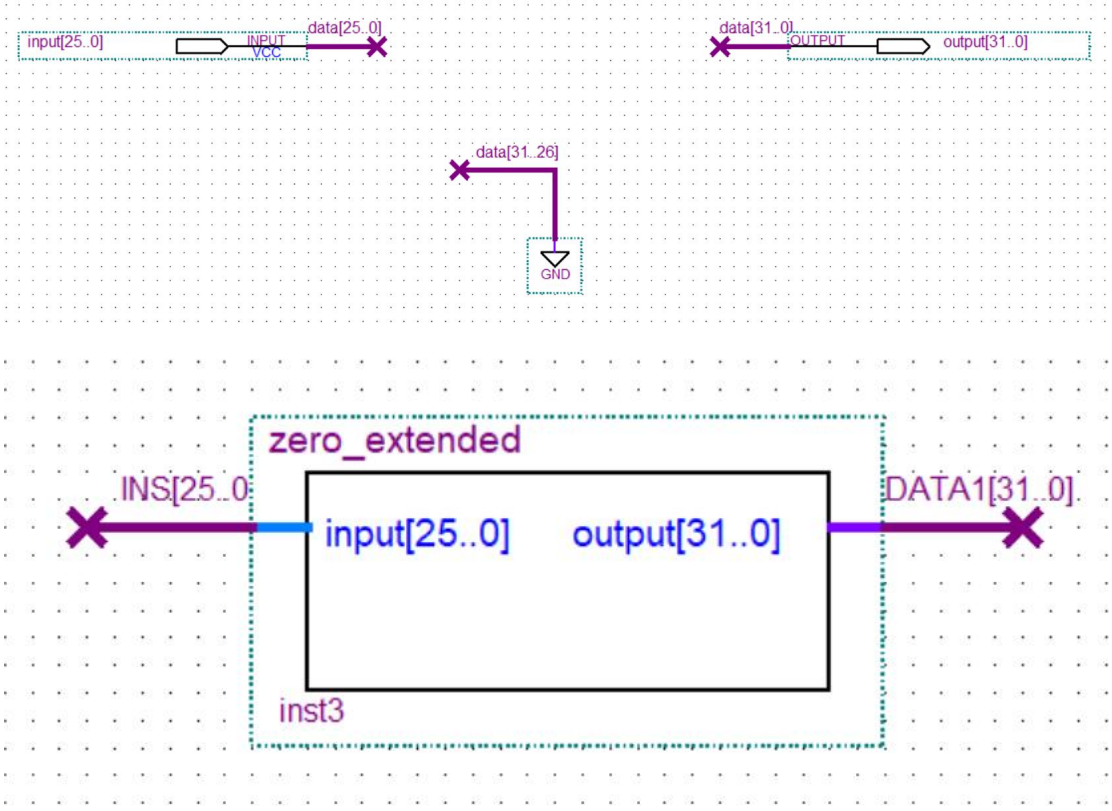
拓展单元分为符号拓展和零拓展。符号拓展单元依照输入数据符号位进行高位填充实现拓展功能，零拓展单元将输入数据高位填充“0”以实现拓展功能。

16_32位拓展：符号拓展部分通过数据选择器，根据输入数据符号位来决定输出数据的高位补“0”还是“1”，如下图：





26_32 位拓展：零拓展部分同过高位补“0”实现，如下图



五、测试结果及分析

5.1 测试方案

我们设计了一个指令周期进行测试，如下：

LW: 101011 00000 00001 0000000010000000 ；将(00000)地址的寄存器作为基址加上 Offset 得到存储器地址，将该地址中的数据存入（00001）地址的寄存器

LW: 101011 00000 00010 0000000010000001 ；将(00000)地址的寄存器作为基址加上 Offset 得到存储器地址，将该地址中的数据存入（00010）地址的寄存器

ADD：000000 00001 00010 00011 reversed(11) ;将(00001)地址的寄存器和(00010)地址的寄存器中的数值相加，再将结果存入(00011)地址的寄存器

SW: 101011 00000 00011 0000000010000010 ;将(00000)地址的寄存器作为基址加上 Offset 得到存储器地址，将（00011）寄存器的地址存入该存储器地址

SHL：100001 00001 00100 reversed(16) ;将(00001)地址的寄存器进行左移运算，计算结果存入(00100)地址的寄存器中

CMP：101001 00001 00010 00101 reversed(11) ;将(00001)地址的寄存器与(00010)地址的寄存器进行 CMP 操作运算结果存入（00101）地址的寄存器中

SUB：100111 00001 00010 00110 reversed(11) ;将(00001)地址的寄存器 data1 与(00010)地址的寄存器 data2 相减，结果 data1-data2 存储在(00110)地址的寄存器中

MOV：111111 00110 00111 reversed(11) ;将(00110)地址的寄存器的内容赋值给(00111)地址的寄存器

J：000010 00000000000000000000000000 ;跳转到指令存储器的 0 地址，从而实现指令的循环执行

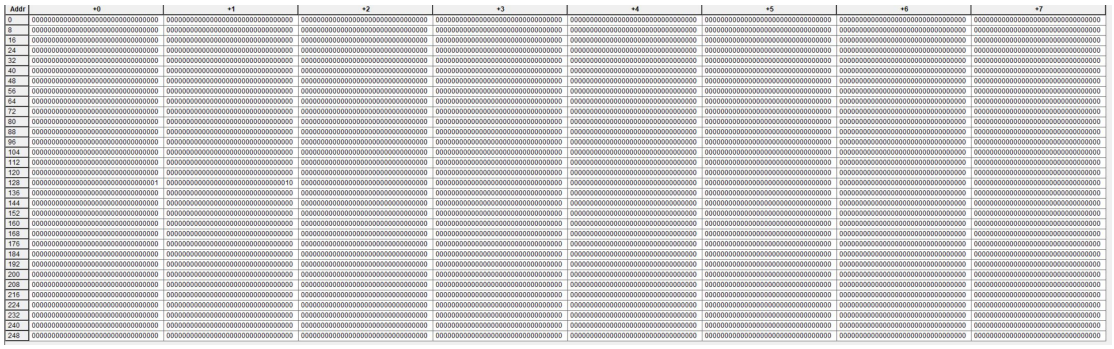
5.2 模块初始值

1. 在指令存储器 ROM 中，我们将我们设计的指令写入.mif 文件中，其内容如下图所示：

Addr	+0	+1	+2	+3	+4	+5	+6	+7
0	00000000000000000000000000000000	10001100000000100000000100000000	10001100000001100000000100000001	00000000001000100011000000000000	10101100000000110000000100000010	10000100001001000000000000000000	10100100001000100010100000000000	10011100010000010011000000000000
8	11111100110001110000000000000000	00001000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
16	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
24	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
32	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
40	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
48	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
56	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
64	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
72	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
80	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
88	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
96	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
104	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
112	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
120	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
128	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
136	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
144	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
152	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
160	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
168	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
176	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
184	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
192	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
200	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
208	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
216	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
224	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
232	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
240	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
248	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000

```
CONTENT BEGIN
        0   :   00000000000000000000000000000000;
        1   :   10001100000000010000000010000000;
        2   :   10001100000000010000000010000001;
        3   :   00000000001000100001100000000000;
        4   :   10101100000000011000000010000010;
        5   :   10000100001001000000000000000000;
        6   :   10100100001000100010100000000000;
        7   :   10011100010000010011000000000000;
        8   :   11111100110001110000000000000000;
        9   :   00001000000000000000000000000000;
        [10..255]  :   00000000000000000000000000000000;
END;
```

2. 在数据存储器 RAM 中，我们在 128 地址位置写入 00000000000000000000000000000001B，在 139 地址位置写入 00000000000000000000000000000010B 作为后续实验测试的两个初始数据。

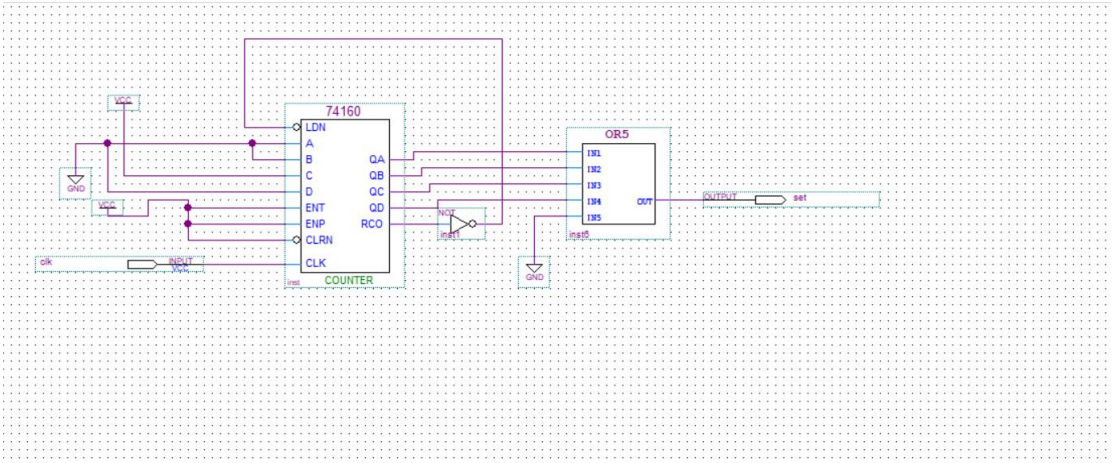


```
WIDTH=32;
DEPTH=256;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

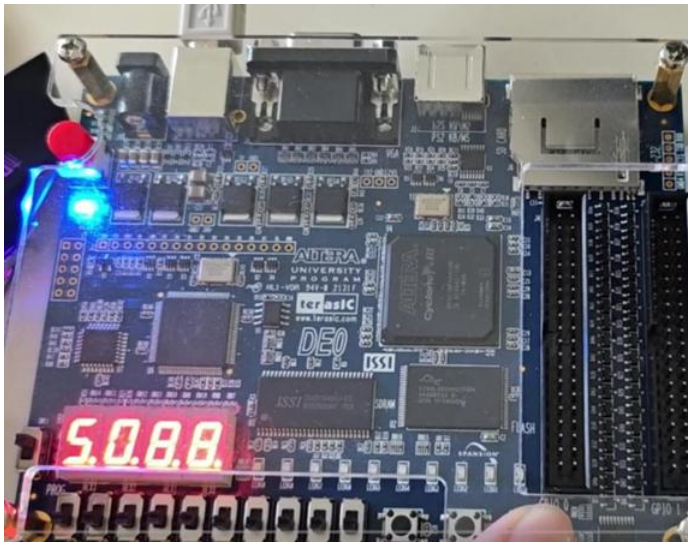
CONTENT BEGIN
        [0..127]  :   00000000000000000000000000000000;
        128   :   00000000000000000000000000000001;
        129   :   00000000000000000000000000000010;
        [130..255]  :   00000000000000000000000000000000;
END;
```

3. 我们在 PC 中利用器件将 PC 的值初始化位零

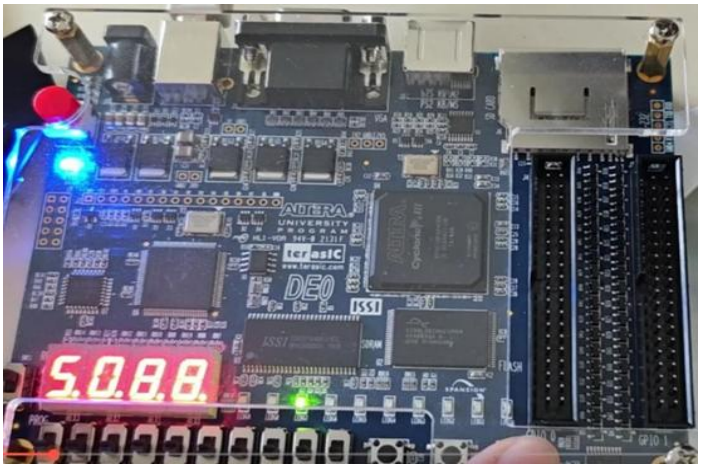


5.3 测试结果及分析

1. 第一条指令为 00000000000000000000000000000000，ADD 指令，初始值为 0，如下图：

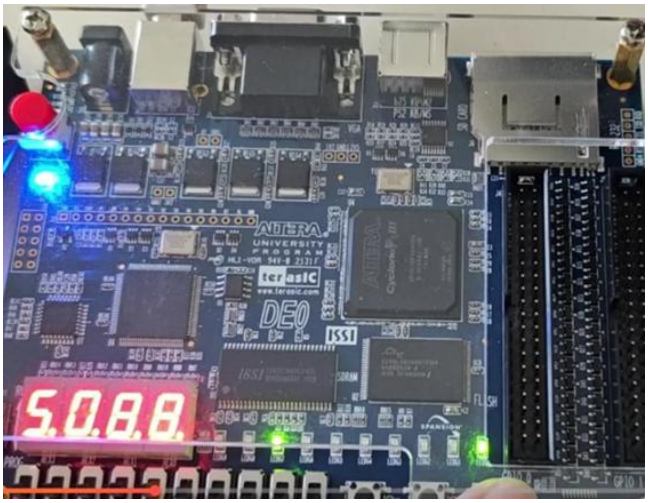


2. 第二条指令为10001100000000010000000010000000，LW指令，显示第一个数据的地址 128；

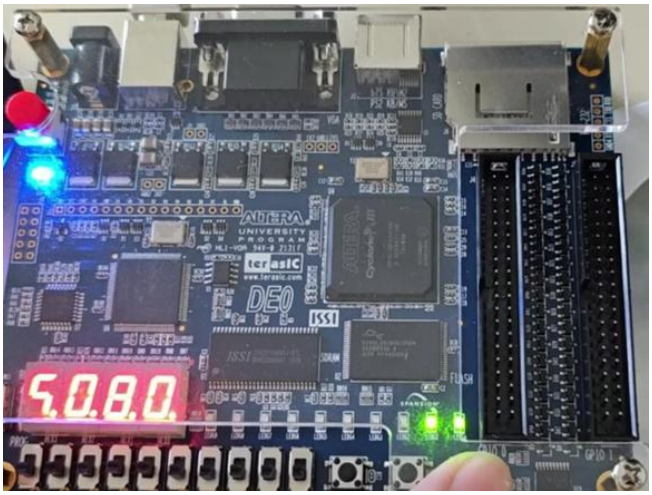


3. 第三条指令为10001100000000010000000010000001，LW指令，显示第二个数据地址

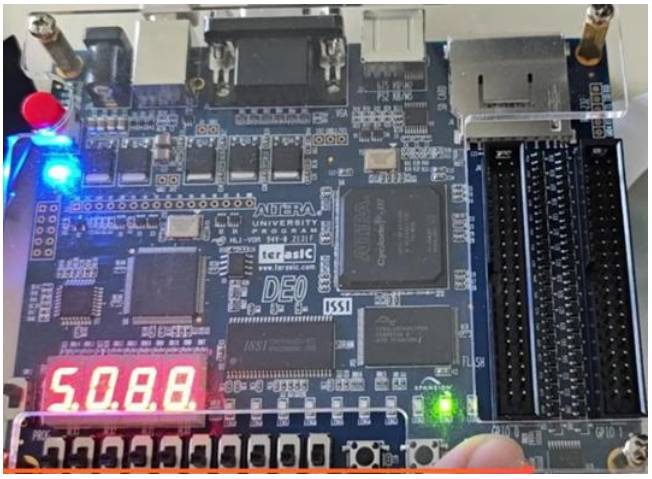
129；



4.第四条指令为00000000001000100001100000000000，ADD指令，由于我们在第一个和第二个数据位置存放了1和2（二进制数1和10），此处输出两个数据之和3（二进制数 11）；

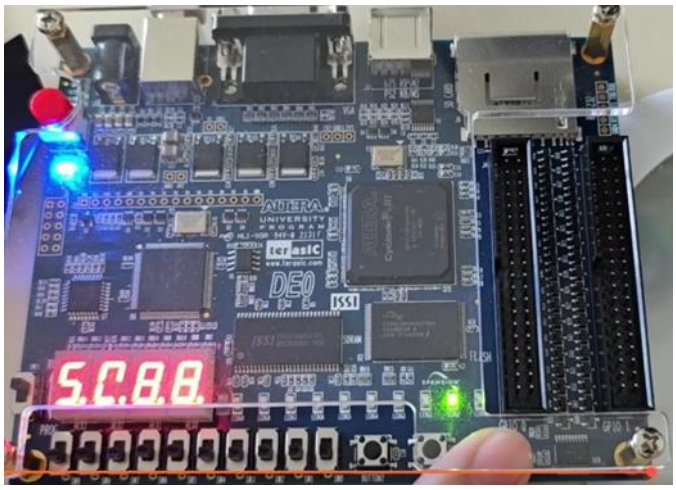


5.第五条指令为10000100001001000000000000000000，SHL指令，二进制数 1 进行左移运算，得到 10；

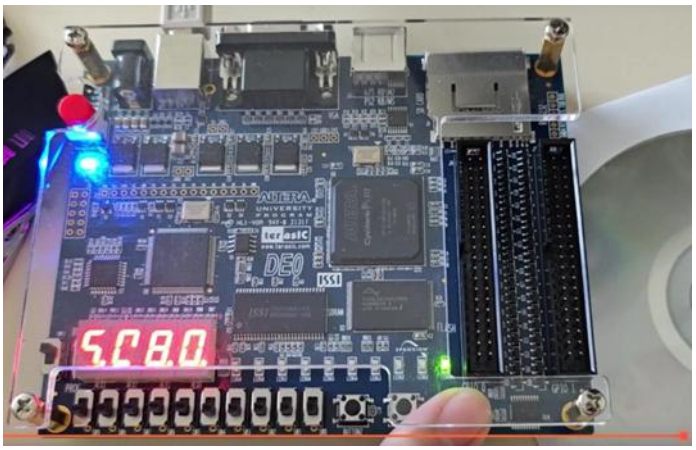


6.第六条指令为10100100001000100010100000000000，CMP指令，将二进制数 1 和二

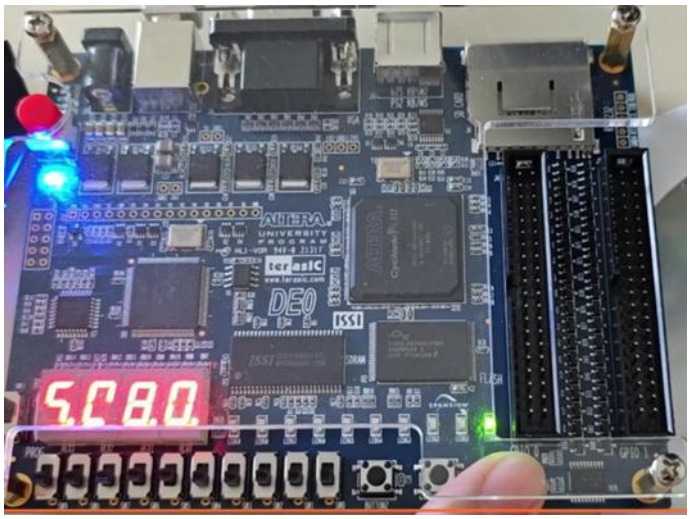
进制数 10 进行比较操作，输出较大的数 10；



7.第七条指令为10011100010000010011000000000000，SUB指令，将二进制数 10 和1 进行相减，得到，结果储存在(00110)；



8.第八条指令为11111100110001110000000000000000，MOV指令将(00110)地址的寄存器的内容赋值给(00111)地址的寄存器输出结果 1；



六、心得与体会

实验设计心得与体会：

经历了这次实验设计，我深刻体会到了硬件控制器的设计与验证过程中所涉及的多方面知识和技能。以下是我从这次实验中学到的重要内容以及这些学习将如何为未来提供帮助：

1. **指令设计与硬件模块设计：** 通过定义和设计单周期指令集，我学到了如何将高级指令翻译成硬件电路。了解了 ADD、LW、SW 等指令的具体功能，以及如何设计 ALU、寄存器堆和存储器等硬件模块以支持这些指令的执行。
2. **时钟设计与时序分析：** 在实验中，我学到了如何合理确定机器周期、节拍与主频，以及如何进行时序分析确保设计的硬件满足时序要求。这对于确保硬布线控制器的稳定运行至关重要。
3. **Quartus II 软件的应用：** 通过使用 Quartus II 软件进行设计、仿真和时序分析，我熟悉了数字系统设计工具的使用。这将在未来的数字系统设计中为我提供强大的支持。
4. **The DE0 board 实验验证：** 在实际的硬件平台上验证设计的可行性，通过观察 LED 灯的状态等方式来确认指令的正确执行，使我更好地理解了硬件设计与实际硬件的连接。
5. **性能评估与设计修正：** 通过记录每条指令的执行时间，我学到了如何评估硬布线控制器的性能。发现问题并进行及时的修正，优化硬件设计，这是持续改进和优化系统的重要步骤。

通过这次实验，我获得了深入的硬件设计经验，更好地理解了计算机体系结构和数字系统设计的原理。这将为我在未来的职业生涯中从事嵌入式系统开发、数字电路设计等领域提供坚实的基础。同时，通过实际操作，我培养了解决问题的能力和动手实践的技能，这对于将理论知识转化为实际应用非常重要。

参考文献

[1] 钱江波. 短学期指导. 宁波大学实验手册，2016/09/10

[2] David Harris, Sarah Harris. Digital Design and Computer Architecture. 2nd Edition. Morgan Kaufmann, 2015.

[3] Patterson, David A., Hennessy, John L. Computer Organization and Design: The Hardware/Software Interface. 5th Edition. Morgan Kaufmann, 2013.

[4] 白中英. 《计算机组成与设计》. 科学出版社, 2019.